

A NEW APPROACH TO MOBILE CLOUD GAMING: REDUCING HARDWARE DEMANDS VIA A NETWORKED EVENT BUS

REECE MACKIE



A REPORT SUBMITTED AS PART OF THE REQUIREMENTS FOR THE DEGREE
OF MSCI IN COMPUTING SCIENCE
AT THE SCHOOL OF COMPUTING, ENGINEERING AND TECHNOLOGY
ROBERT GORDON UNIVERSITY
ABERDEEN, SCOTLAND

April 2025

Supervisor Dr. Kyle Martin

Abstract

Mobile gaming is the most popular form of gaming, despite the unique challenges posed by the form factor, such as: computational limitations, power constraints, limited control schemes, and display sizes. To combat the lack of computational power, many are turning to cloud gaming to bring large titles to the mobile games market by streaming games that are running in a data centre.

This project proposes a new framework for developing hybrid cloud mobile games, architected around a networked event bus used for synchronising game state between the local client and remote server. It enables gameplay systems to be designed in a manner which allows them to be executed on either peer. This makes it possible to dynamically offload these systems to optimise the game's performance while ensuring efficient use of locally available resources. To demonstrate the capabilities of this framework, a test game was built to both functionally test the framework and serve as a performance benchmark to determine if this approach can reduce the performance requirements of a game.

Functionality testing demonstrates that this framework can provide game state management and synchronization and successfully facilitates the creation of gameplay systems which can be executed locally and on a remote server. Despite this, the framework did not perform as expected during performance testing, on a more capable mobile device it was able to improve computational load on a high-end mobile device, however performance deteriorated on a low-end device when compared to the game running fully locally.

Acknowledgements

I sincerely thank my supervisor, Dr Kyle Martin, for his invaluable guidance and feedback during this project. I would also like to thank the rest of the SoCET staff and my classmates for making my time at university fulfilling. Finally, I would like to thank my family, partner and friends for their unwavering encouragement throughout my studies at university.

Declaration

I confirm that the work contained in this MSci project report has been composed solely by myself and has not been accepted in any previous application for a degree. All sources of information have been specifically acknowledged and all verbatim extracts are distinguished by quotation marks.

Signed Reece Mackie Date 29/04/2025
Reece Mackie

Contents

Abstract	ii
Acknowledgements	iii
Declaration	iv
1 Introduction	1
1.1 Project Overview	1
1.2 Motivations and Objectives	1
1.3 Chapter List	2
2 Background Research	3
2.1 Video Game Lag	3
2.2 Cloud Gaming	4
2.2.1 Game Availability, Ownership and Subscriptions	4
2.2.2 Network Limitations	5
2.3 Current Solutions	6
2.3.1 Mobile Game Optimization	6
2.3.2 Cloud Game Streaming	7
2.3.3 Edge Computing	8
2.3.4 Hybrid Streaming	9
2.3.5 Performance Measurements	12
2.4 Conclusion	13
3 Design	14
3.1 Distributed Compute with ‘Systems’	14
3.2 Synchronising Game State	15
3.3 System Balancing	15
3.4 Test-Bed for Evaluating the Framework	15
3.5 MonoGame Challenges	16

3.6	Technologies, Tools and Resources	16
4	Implementation	18
4.1	Project Structure	18
4.2	Framework	18
4.2.1	Game Structure	18
4.2.2	Registries	19
4.2.3	Defining Game Systems	19
4.2.4	Dynamic System Balancing	20
4.2.5	The Event Bus	21
4.3	Test Game	22
4.3.1	Game Systems	22
4.3.2	Link Between Virtual Level and Unity Scene	24
4.4	Automated Performance Benchmark	25
5	Evaluation & Testing	27
5.1	Testing Methodology	27
5.2	Functionality Testing	28
5.3	Performance Testing	29
5.4	Framework Evaluation	33
5.4.1	Event Bus Pitfalls	33
5.4.2	Packets Stuck in Network Queues	33
6	Conclusions and Future Work	34
6.1	Conclusions	34
6.2	Personal Reflection	35
6.3	Future Work	35
7	Bibliography	37
A	Project Log	41
B	Source Code	44
C	All Results	45

List of Tables

2.1	Comparison of hybrid-cloud streaming solutions	11
4.1	The three different benchmark types and their purpose.	25
5.1	Devices Used for Performance Evaluation	28
5.2	Frame rate averages and 1% lows for all benchmarks	29
5.3	Pearson correlation coefficients for event counts across the local and re- mote sides of the benchmarks.	31

List of Figures

2.1	Depicts player reaction time and accuracy in a low and high latency environment. Source: Halbhuber et al. 2023	4
2.2	Five characteristics of a successful cloud gaming service. Source: STL Partners (Osborne 2023)	5
2.3	Overview of Steam Remote Play. Source: Steam 2024	8
2.4	The median latency for covered users from 3 US ECS zones. Source: Choy et al. 2014	9
2.5	Data flow of the proposed hybrid model. Source: K. L. Chan et al. 2017	10
2.6	Alternative hybrid streaming approach. Source: Nan et al. 2017	11
3.1	Distributed client-server model, showing systems being executed on either side and the game state being synced over the network.	14
5.1	Average frame rate over time on the client during the local only benchmark for both devices.	29
5.2	Average frame rate over time on the client during the local only benchmark for both devices.	30
5.3	Average frame rate over time on the client during the standard benchmark for both devices.	30
5.4	System count over time for the standard benchmark on both devices. . .	31
5.5	System run time drops at the 50-60 second mark in all three runs while running locally on the iPhone.	32
5.6	System run time over time during the local only benchmark on the Android VM.	32

Listings

4.1	Game System Interface	20
4.2	Dynamic Offloading Logic	21
4.3	AbstractEvent Class	21
4.4	Example weapon item definition	23
4.5	OnEntitySpawn override on ClientLevel, passing the information to the Client Level Manager.	24

Chapter 1

Introduction

1.1 Project Overview

This project proposes a hybrid cloud gaming framework to facilitate sharing the game's load between the local client and a remote server to increase compatibility with lower-end devices and extend battery life. This framework will facilitate the execution of independent gameplay systems on either peer and synchronise the changes to the game state over the network.

To achieve effective network synchronisation, the framework uses a networked event bus. Events are fired whenever the game state should be updated (e.g., position change, level up, or item collected). All events are written in a network-serializable manner to facilitate this functionality. The event bus helps to simplify game logic as handling an event works the same, regardless of whether the event was fired locally or remotely.

To test and evaluate the framework, a test game will be built based on Vampire Survivors. This serves as both a benchmark to discover whether or not this cloud gaming strategy is effective, and a tool to demonstrate the frameworks capability to support the kind of video game that may be played on a mobile device.

1.2 Motivations and Objectives

This project aims to build and evaluate the proposed framework to determine if this strategy is an effective way to approach hybrid compute. If this framework successfully completes these objectives, it could be refined and used to build a fully-fledged title which features hybrid compute functionality in a way that respects game ownership and preservation. If it is not capable of fulfilling all of the objectives, it may inform

future strategies for bringing games to lower-end devices.

This project has the following objectives:

1. **Evaluate hybrid computation for video games** to determine if it is an effective cloud gaming strategy.
2. **Ensure efficient use of computational resources**, maximising local resources where possible.
3. **Enable lower-end devices to play** normally inaccessible games.
4. **Reduce the energy consumption of mobile games**, leveraging remote resources to increase play time on battery power.
5. **Protect game ownership rights** by ensuring the game is accessible fully offline.

1.3 Chapter List

Chapter 2 Background Research. This chapter provides a review of literature relevant to this problem.

Chapter 3 Design. This chapter establishes the design of the framework.

Chapter 4 Implementation. This chapter discusses how the framework and testing game was implemented, and how it works.

Chapter 5 Evaluation & Testing. This chapter tests both the function and performance of the framework and game, and evaluates the outcomes of the testing.

Chapter 6 Conclusion. The conclusion compares the evaluation results against the project objectives, provides a personal reflection and some future works.

Chapter 2

Background Research

Mobile gaming is significantly more popular than gaming on a PC or Console (S. Chan [2017](#)), projected to generate approximately \$98.7bn USD world-wide in 2024 (Statistica [2024b](#)). Despite the highly profitable nature of mobile gaming, developing for mobile platforms have unique challenges such as thermal/power budgets, unique control schemes and limited display size (Beets [2021](#)). As such, developers must adapt to overcome these challenges to ensure mobile games are enjoyable and profitable. This review will investigate strategies that can be employed to improve mobile video game performance, through optimisation and the utilisation of cloud resources.

2.1 Video Game Lag

According to a report published by EdgeGap in 2022 (Edgegap [2022](#); Rees [2022](#)), 30% of mobile gaming respondents reported that they have experienced lag in some way while playing mobile games. 34% of gamers say that lag causes them to quit the game immediately and 42% of players avoid playing laggy games as often. This shows that there is a significant margin for improvement in mobile video game optimisation that could result in higher active players. 31% of respondents said they'd be satisfied with sacrificing some graphical fidelity for improved performance, this can generally be acceptable as mobile devices tend to have smaller displays that are incapable of displaying the finer details to the user (Beets [2021](#)). This study is supported by the findings of (Halbhuber et al. [2023](#)), where player performance and experience were negatively impacted by lag (in this paper – lag was introduced by artificially inducing network latency in the router).

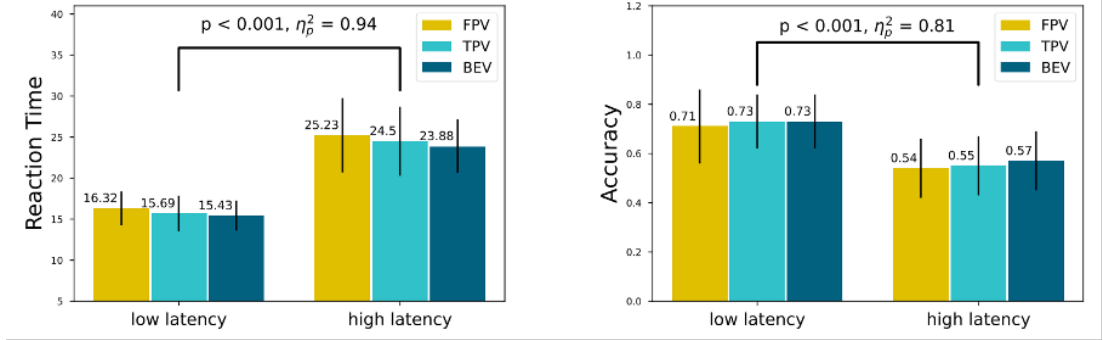


Figure 2.1: Depicts player reaction time and accuracy in a low and high latency environment. Source: Halbhuber et al. 2023

2.2 Cloud Gaming

Recently, a new model for playing games has emerged to run larger video games on mobile platforms without needing to reduce the game’s graphical and computational requirements: Cloud Gaming services. Cloud gaming leverages existing Cloud Computing technologies to enable video game software to be accessed through thin-client software on a player’s device (Shea et al. 2013). Cloud gaming is an evolving market worldwide and is estimated to generate \$6.91bn USD in 2024 (Statistica 2024a), attracting the interest of researchers, investors and gamers alike. Many commercial services already supply cloud gaming today, most notably: GeForce Now (NVIDIA), Xbox Cloud Gaming (Microsoft) and PlayStation Plus (Sony) (formerly known as PS Now).

2.2.1 Game Availability, Ownership and Subscriptions

There are a handful of caveats surrounding cloud gaming. For starters, game availability is an important factor for cloud gaming subscriptions. For players to feel they are getting value from their subscription, the service must offer access to games that are relevant to their interests. STL Partners (Osborne 2023) suggest that this is one of the reasons that Google Stadia (Google) failed to attract the attention of a wider audience.



Figure 2.2: Five characteristics of a successful cloud gaming service. Source: STL Partners (Osborne 2023)

This pivot to subscription-based access to games, as opposed to game ownership (technically speaking, game purchases are licensed to the player via a game store) does have ramifications for game access. For example, what happens when the service provider goes out of business? Or a game reaches the end of its life and can no longer be played? In the case of Stadia, it meant that gamers were refunded for every game they purchased from Google, however a startup would not have the funds to do the same and access would be lost. This possibility is a threat to game preservation, as well as maintaining access to titles purchased by gamers. However, it appears that this is following in the shift in Music and Video digital ownership (Lindzon 2024).

As noted by Gameopedia (Balasubramanian 2022), live-service games depend on digital content delivery to be able to serve content to players; with cloud gaming it can become easier for developers and publishers to manage rollouts of these updates. If a game was cloud-exclusive, prevent players data mining for unreleased information that plague live-service game releases – for example the game “Destiny 2” regularly suffers leaked content due to data-mining efforts (The NPC Jarvis 2024). Naturally this approach is harmful to game preservation and ownership, but the benefit of helping players avoid leaked information is also a considerable boon to player and developer experience.

2.2.2 Network Limitations

Another cloud gaming caveat is the network requirements. In an analysis comparing Google Stadia, GeForce Now and PlayStation Plus game streaming (Domenico et al. 2020). Stadia and GeForce now were measured to have a bit rate of 45 Mbit/s while streaming a 4K video game, which is much more substantial when compared to a 4K Netflix video stream, which measures 15 Mbit/s. According to Cable.co.uk 2024 Broadband Speed League, the average internet speed worldwide as of their 2024 survey is

55.38 Mbit/s. 29 Mbit/s is poised comfortably below the world-wide average; however, this becomes more of a challenge once you consider a household with more than one person attempting to use another bandwidth-intense service.

Another issue that plagues Cloud Gaming services is latency (which can be measured as the Response Delay, or RD (Chen et al. 2011)). With a long RD, input commands can take a longer time to be processed, and the results are displayed to the player much slower than expected. This delay can be caused by a handful of factors, but the most common is the network round-trip time (RTT). This is caused by the player having a poor connection to the internet (slow hardware, slow uplink etc.), or by a large distance between the player and the game server they are using (Cloudflare 2024). Having high network latency can result in noticeable lag for the player, which as discussed already results in a poor experience and thus poor player retention.

2.3 Current Solutions

2.3.1 Mobile Game Optimization

One strategy for making mobile video games run for longer and with better performance is to optimise the game specifically for mobile targets. This is the most obvious choice for most developers, as this is something that can be solved in-house without the need for a partnership with a cloud gaming provider, or a first-party cloud gaming solution. Unity 2023 does urge developers to take caution when optimising mobile games because optimisations tend to introduce additional complexities to video game software which can be timely and costly to maintain and tend to introduce non-trivial bugs. They also recommend that developers use a performance profiler to identify exactly which code paths are causing significant performance degradations to identify where optimisations are best to be made. (Koulaxidis and Xinogalos 2022) agrees with this, recommending that developers should be looking at the profiler to identify high-cost parts instead of taking shots in the dark.

Unity 2023 recommends determining what the game's frame budget should be to aid with optimisation. A game running at 30fps cannot exceed 33.33ms per frame (otherwise it is running slower than 30fps), and a 60fps game cannot exceed 16.66ms, however this is the maximum frame time. Unity recommends only utilising 65% of this frame time to allow cooldown between frames and avoid thermal throttling the device, for 30fps this is 22ms per frame and 60fps is 11ms. It is permissible to boost outside these budgets for short periods of time for loading sequences, cutscenes and so on.

Technologies like Adaptive Performance (Unity 2023) can further assist with thermal

control, by adapting different parameters in the game (for example a dynamic level-of-detail) based upon the thermal load of the device, to ensure a consistent frame time is maintained. The use of on-device thermal data is a good idea to help fit the game into the frame budget if there is a temporary high load. Both Prakash et al. 2016 and Choi et al. 2022 present more involved strategies for thermal management at the operating system level, governing the CPU and GPU in a such a way that the entire SoC's (System on a Chip) power (and thus thermal output) are constrained. Strategies like these are promising for improved gaming performance which adapts to the system's capabilities.

2.3.2 Cloud Game Streaming

Cloud gaming services work by hosting servers equipped with GPU's and plentiful memory and compute (sometimes contained within virtual machines) which are accessed from the client application. The client application requests a session and once a server is allocated, the client sends inputs (keyboard, mouse, controller etc.) to the server, and the server routes these inputs to the video game software, and then captures and streams the video output back to the client (Huang et al. 2013). This solution enables video game developers to target the PC platform, while the game is accessible on console and mobile targets too. This is the approach taken by most commercial game streaming services, such as: GeForce Now, Stadia, PlayStation Now, GamingAnywhere (Huang et al.) and OnLive 2010 (now defunct).

As discussed by Huang et al., users expect high quality video and low response times – which can prove to be especially challenging. As mentioned in the background, a network analysis of three popular cloud gaming services (Domenico et al. 2020) found that GeForce Now and Stadia both had a measured bit rate of 29 Mbit/s for 1080p gaming – however PlayStation Plus (Formerly PS Now) was measured to have a considerably lower bit rate of 13 Mbit/s using its proprietary protocol. PlayStation acquired both OnLive and Gaikai 2009 and applied their technologies to create their game streaming solution (Orland 2015). Thus these performance improvements are proprietary but prove that there are strategies available to drastically reduce the bandwidth requirements of cloud game streaming.

There are also methods of game streaming that are not provided as a service, for example: Sunshine (LizardByte), Steam Remote Play (Steam), and StreamMyGame (Tenomichi Ltd) (now defunct). These allow a user to utilise their existing hardware from another location, such as streaming a game from a desktop PC to a mobile phone, laptop or TV. These tend to utilise the same technology as the cloud streaming services, however are bounded by the user's own network and hardware – this tends to work well

within the home network but would be less effective for gaming from a distance where a cloud service potentially has servers closer.

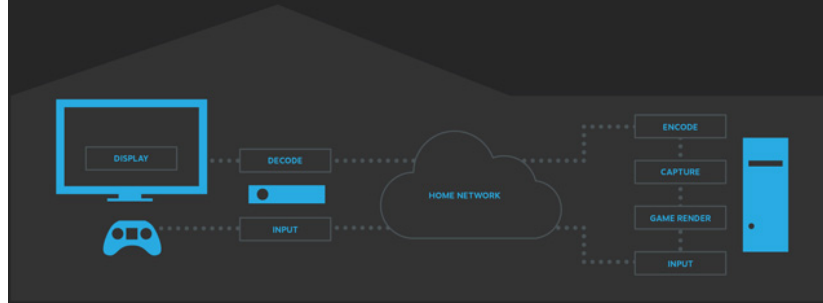


Figure 2.3: Overview of Steam Remote Play. Source: Steam [2024](#)

An alternative model for cloud game streaming is instruction-based streaming. Eisert and Fechteler [2008](#) present a system which captures OpenGL and SDL API calls and streams these back to the thin client, which proceeds to execute these streamed commands. This enables all the computation for the video game to be performed on the server, however the thin client is responsible for rendering the final game output and capturing inputs. Ultimately the result is like that of video streaming, however it reduces the bitrate by 80% and lowers the latency due to the removal of video encoding + decoding delays. This approach does have the downside of relying on the thin-client’s GPU capabilities, however if the device is capable enough the bandwidth and latency reductions are compelling.

2.3.3 Edge Computing

Edge computing is a recent shift in cloud computing services, which places compute and storage resources at “the Internet’s edge”, which places resources into (near) local proximity to endpoint devices (Satyanarayanan [2017](#)). This proximity between cloud resources and client devices allows for much lower latency compared to having resources in a regional datacentre – or even in a single datacentre. Edge computing technologies has traditionally been used by technologies such as Content Delivery Networks (CDNs) and in recent years, entire web applications can be deployed to the edge for improved loading and response times. Service providers such as Vercel [2024](#) and Netlify [2018](#) are providing edge deployments for web apps.

Previous studies have declared that the maximal threshold for network latency before it begins to affect the user’s gameplay experience is 80ms (Choy et al. [2012](#)). This same study measured the average latency between 2,504 US clients in three AWS datacentres across the US and discovered that each location was only able to serve 70% of its users

with acceptable latency ($<80\text{ms}$).

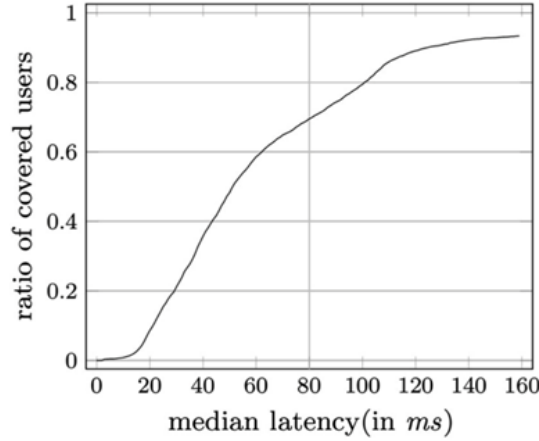


Figure 2.4: The median latency for covered users from 3 US ECS zones. Source: Choy et al. [2014](#)

Ultimately this study determines that the use of hybrid infrastructure would be more beneficial, utilising centralised cloud compute where possible as opposed to exclusively edge servers. This allows a reduction in cost while still being capable of improving coverage of the top 30% of end-users who are not within acceptable range of a datacentre.

Choy et al. [2014](#), an earlier paper by the same researchers, presents the EdgeCloud platform. EdgeCloud is an example of a hybrid-cloud architecture for game streaming, which uses central datacentres to cover its main users and combines this with capable peers closer to the edge, which helps to bolster service coverage using capable hardware. The paper suggests that idle games consoles and home computers are reasonable candidates to become peers, and that the network should provide incentives for any entity contributing compute. The researchers discuss how this may still be a significant challenge, as it is imperative that there is enough capacity to support all clients at once, to avoid queueing for service.

2.3.4 Hybrid Streaming

K. L. Chan et al. [2017](#) propose a new strategy, a combination of traditional video-streaming and instruction-streaming (Eisert and Fechteler [2008](#)) strategies. This involves interception of the graphics API calls, similarly to the instruction-streaming method, however these instructions are then separated based on depth (z-value), effectively separating the commands into background and foreground layers. The foreground commands are streamed to the client for rendering on-device, however the background is rendered on the server and streamed on video. This enables the foreground elements

to appear crisp, while the background can suffer some lower quality depending on network condition. The researchers discovered this worked well for their demo application, however, are unsure on how applicable this approach may be to other genres of game. They also determined that their improvement in graphics quality is not significant – this suggests that the video-streaming strategy is difficult to beat from a quality perspective, and instead should perhaps be considered as the standard for incapable thin clients, and instruction-streaming should be employed in scenarios where the thin client is more capable, and thus can lower the required bitrate necessary to play.

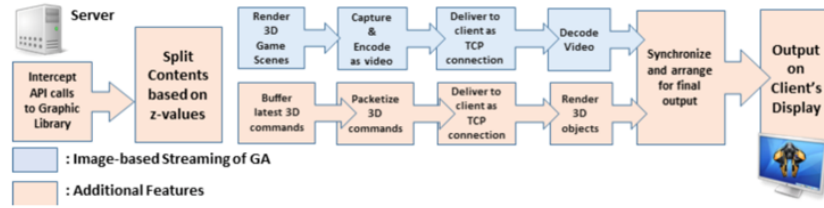


Figure 2.5: Data flow of the proposed hybrid model. Source: K. L. Chan et al. 2017

Nan et al. 2017 propose an alternative hybrid strategy, instead of capturing GPU commands they capture the models and textures of the completed scene and stream these to the client instead. These resources are collected in a buffer on both the server and the client, and once sufficient data is available to render the scene locally, the server will stop encoding video and instruct the client to begin local rendering. The initial video stream allows for instantaneous visuals when the game first loads, or the scene changes while the relevant graphics resources are being collected in the buffer. This contrasts with the approach taken by K. L. Chan et al. 2017 as they continued to stream video alongside the foreground commands. This approach also considers dynamic switching between hybrid and video-only, depending on the client capabilities – something that was not covered by Chan et al. Furthermore, they determine the correct parameters for the graphics and video streams using Delay-Rate-Distortion Optimisation. This maintains visual fidelity while synchronising game objects to the client.

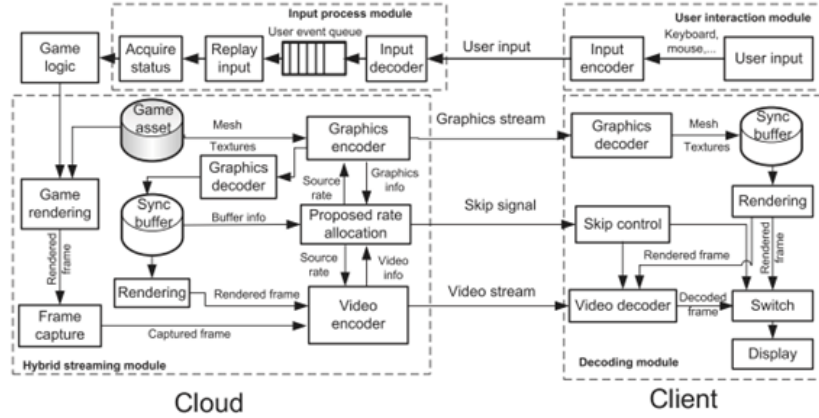


Figure 2.6: Alternative hybrid streaming approach. Source: Nan et al. 2017

There are a range of scenarios discussed in Table 2.1 demonstrating where either solution provides the best experience, and of note is a scenario which would benefit from a combination of these two solutions.

Scenario	Optimal Solution
A graphics intensive game running on an adequately capable client device, with moderate quality network.	The solution proposed by Nan et al. is the most suitable as it enables the client device to fully utilised while keeping overall network utilisation down, compared to Chan et al. Optimising visual quality with network availability is an added strength.
A graphics intensive game running on a less capable client device, with good quality network.	Chan et al. presents the most suitable solution, off-loading the background graphics while rendering foreground elements on the device will improve local-rendering performance, while taking advantage of the improved network quality, covering the shortfall in local graphics capability.
A graphics intensive game running on a less capable client device, with moderate quality network.	A combination of both solutions would be ideal here. Being able to switch between full video streaming, depth-based video streaming and full on-device graphics would enable the game to adapt to changes in device capability and network quality – netting the best performance for the user.

Table 2.1: Comparison of hybrid-cloud streaming solutions

This outlines that both solutions have varying strengths, however there are potential further improvements to game performance if these strategies were combined in such a way that they can dynamically adapt to the network and client device capability and conditions.

2.3.5 Performance Measurements

There are several ways to measure video game performance. Firstly, you have the frame rate (measured in Frames Per Second, FPS). This measure describes how many frames of video the game is producing per second (Koulaxidis and Xinogalos 2022). In many cases, this is also the local update rate (measured as Updates Per Second, UPS) however in some scenarios a game may be multi-threaded, and update at a different rate to the frame rate. These figures are generally adequate for testing fully local video games, as they are highly indicative of how well the game is performing on the hardware. These figures are not as appropriate for cloud streaming however, as update rate is always anticipated to be constant on the cloud hardware but in some conditions the frame rate is still a valid metric. For example, measuring how many frames are being delivered to the thin client per second over the network.

Chen et al. 2011 propose a method for measuring the latency of cloud gaming systems. The goal is to measure the system’s Response Delay (RD).

$$RD = ND + PD + OD$$

- **Network Delay (ND)** - the time from command to response, a.k.a. Round-Trip Time.
- **Processing Delay (PD)** - time between the server receiving the command and the result frame
- **Playout Delay (OD)** - the time between the client receiving the frame, and the frame being decoded and presented.

This method is presented in such a way that it can be applied to both open and closed systems without the need to modify their internals. This measure is effective for all video-based cloud streaming solutions, both hybrid and otherwise. The round-trip time was also used as part of measurements in the study by Domenico et al.

The Domenico et al. 2020 study also measures the bit rate of the overall solution. This gives a vague measure of how much data is being sent to the client per second for the audio-video (AV) stream. While this does not help to describe the contents of the stream, it does help to assess the accessibility of the service, when compared to the

maximum speed of the network the service is being accessed from – for example limited 3G networks compared to Gigabit Broadband. This measure should be used to assess how accessible the service is over limited networks, as opposed to a performance metric.

2.4 Conclusion

In conclusion, there are a vast number of strategies to improving mobile game performance. Mobile games can be designed and optimised specifically for the low computational resources available on-device and can be made to dynamically respond to the thermal load to ensure consistent gameplay quality. For larger titles that would need to be significantly simplified, or too much effort to optimize and reduce for the mobile platform, cloud gaming solutions are much more attractive. Hybrid streaming also enables more capable clients to provide higher quality graphics, while still reducing the overall cost of running the entire game on the thin-client, and this can be done dynamically based on the client’s capabilities. This allows developers to deploy games with consistent hardware requirements while accessing the expansive mobile gaming market. These strategies suffer varying challenges, such as network consumption and game library challenges.

This research has identified one opportunity for further development – dynamic hybrid compute. All the discovered works focus on methodologies for optimising video streaming, however, there appears to be no work to hybridise the computational requirements of video games. This can be done in a dynamic fashion to adapt to each client-device’s capabilities. Potential benefits of such could solve some of the limitations of cloud streaming solutions, such as game ownership and play in absence of internet access. As such, a list of requirements for this system will be created to form a proof of concept.

Chapter 3

Design

3.1 Distributed Compute with ‘Systems’

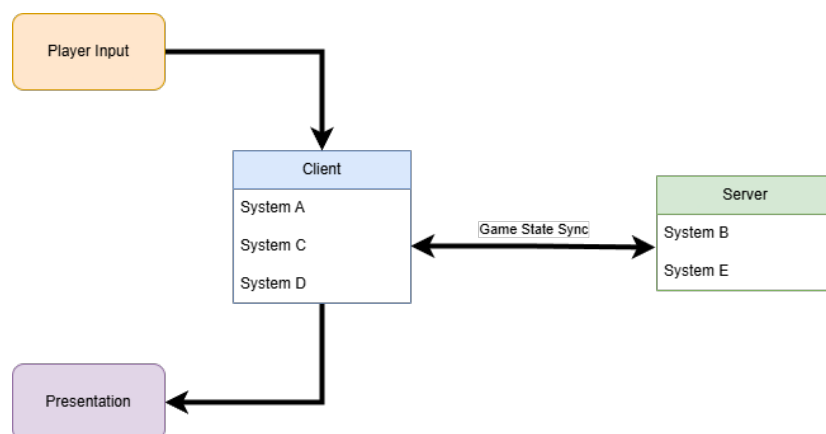


Figure 3.1: Distributed client-server model, showing systems being executed on either side and the game state being synced over the network.

To enable distribution of game logic across both the client and server resource, gameplay logic must be abstracted into a form which is composable and can be moved between the two resources with minimal delay. Initially, a ‘Task’ based system was considered. For each task that can be executed on either the client or server, it would be bundled (along with its necessary parameters/state) and sent to a ‘Task Executor’ which would either execute the task locally or send it for remote execution on the server. The tasks would be written and kept in a shared library, making sure that the task logic is consistent on both the client and server - any discrepancy in shared library version would result in unexpected behaviour that is potentially difficult to identify. This system was discarded early into the design phase however, as this is a poor abstraction over game logic. Game

logic tends to be composed of individual components (or ‘Systems’) which are executed every frame, rather than invoked on occasion.

To more appropriately address the needs of game logic, the concept of ‘Systems’ was considered instead, derived from the concept of a System in the Entity-Component-System (ECS) pattern. In ECS, Systems act using a view into the entities and components of the world in a way akin to a database query, reading and mutating the game state. This concept will be adopted for the hybrid compute system as a strategy to enable or disable systems on the client or server, and because they act on the game state they do not need to encapsulate the world’s state. By keeping the world in sync on both sides, even an abrupt disconnection will not disturb the operation of a system, as all of the necessary state will still exist.

3.2 Synchronising Game State

To ensure the game state is consistent on both the server and the client, any time the game state is modified it needs to be updated locally and consequently send the changes over the network to be applied on the other side. For this to be possible, both the client and the server need to have a common way to represent the game world, which can be updated in an identical manner from either side. Ideally there should also be a strategy for when the game state inevitably becomes de-synced due to packet loss or other network anomalies, but this is out of scope for this project.

3.3 System Balancing

Another requirement for the framework is the capability to dynamically shift the game’s systems between the local and remote resource, prioritising the performance of the game locally. This can also aim to keep ”system runtime” (the amount of time spent each second running gameplay systems) in a reasonable range, to ensure local resources are still used where available. The systems should be able to switch to the remote immediately and without minimal gameplay interruption to ensure the player is unaware of the change. This could be extended in future to consider further telemetry, such as: processor temperature, battery level, and network quality.

3.4 Test-Bed for Evaluating the Framework

To test the hybrid compute framework, a game will be developed to demonstrate its capabilities and to gather performance metrics. This game must be sufficiently complex

such that it has a demonstrable performance impact, and makes full use of the framework's capabilities, these include but are not limited to: system offloading, custom tasks, and pausing on both sides.

Taking these factors into consideration, building a game similar to Vampire Survivors (poncle 2022) would fit the testing requirements. This is a game which involves a large number of moving entities (enemies and projectiles) in the world simultaneously. It also features a number of systems which can be clearly defined and separated (Physics, Enemy Navigation, Damage, Particles, and many more). This would be a good fit for the performance requirements of the game, as each entity added will increase the amount of work each task must perform per frame.

3.5 MonoGame Challenges

Initially, this project was going to be built using MonoGame¹. This was because it provides less built in functionality which would not have been useful, and grants more freedom over the project structure.

Unfortunately, many significant challenges were faced when targetting iOS. First, MonoGame's tooling is not fully compatible with Apple Silicon - it required an x86 version of .NET and the installation of Wine to run shader compilers designed for Windows. Secondly, it became clear that most public libraries that provide extended functionality (such as UI frameworks) did not support Ahead-of-Time compilation, meaning these libraries could not be used.

Given Android represents 71.9% of the mobile OS market and iOS accounts for 27.68% (Statcounter 2025), it is important for the framework to support both platforms. As a result, the decision was made to use Unity instead to ensure the project could target both operating systems within the time constraints.

3.6 Technologies, Tools and Resources

- **Unity:** Unity will be used to build the game and framework. It will be primarily responsible for rendering and input.
- **LiteNetLib²:** A C# framework that provides UDP networking through a high-level API, featuring packet serialization and reliable packet transmission options.

¹<https://monogame.net>

²<https://github.com/RevenantX/LiteNetLib>

- **C#/.NET**: The project will be developed in C#, a language very commonly used in video game development.
- **JetBrains Rider**: The chosen IDE for writing C# code - provides good debugger tooling and compatibility with the ecosystem.
- **Xcode**: The required IDE for iOS development. This will be used to deploy to iPhone during performance testing.
- **Jupyter Notebook**: Used to process, visualise and analyse the performance benchmark results.
- **Kenney's Royalty Free Assets**³: The source of all visuals used for the test game.

³<https://kenney.nl>

Chapter 4

Implementation

4.1 Project Structure

For simplicity, the framework and game were developed within the same Unity project. In `Assets/Scripts` there are four subdirectories:

- **Framework** contains the reusable framework functionality, with no game-specific logic.
- **Common** contains the game logic which is shared with the client and server.
- **Client** contains the client-specific behaviours and integration with the Unity scene for rendering.
- **Server** contains the server-specific logic, such as the connection manager and creation of levels for incoming connections.

4.2 Framework

4.2.1 Game Structure

The framework defines an abstract class, `AbstractHybridGame` which contains all of the necessary components for the hybrid compute functionality, game state management, the event bus and the networking functionality. Games that use this framework are expected to extend this class to include additional event types, systems and properties specific to the game - this will be covered in more detail in [Section 4.3](#).

The most basic form of game state is the list of entities. Each entity must derive from `AbstractEntity`, this provides basic properties such as identities, positions, size,

movement speeds, entity layers, and a utility for generating random numbers. These entities also include virtual methods to enable custom entity types to transmit additional data over the network (e.g. player's level and items). All entities must have an entity type which must be stored in the registry (see Section 4.2.2). The entity type provides a factory to re-create the entity on the other side of the network, and is useful for differentiating entities without runtime reflection.

Data can also be stored globally on the Game class, synced either with events or a special sync packet sent every tick. For example, the test game uses the on-tick packet to transmit the current game time to keep the server in sync. This approach enables the game to decide how data is stored, and how to synchronise data.

Finally, the base game class provides a utility to serialize and deserialise the entire game state. These are used during for initial configuration packet, sending all of the game's state to the remote. While connecting, the event bus can be configured to queue new messages, ensuring that the server can be rapidly re-synchronised once the level is established remotely. The specifics of connection handling is left up to the game implementation.

4.2.2 Registries

The framework makes extensive use of the registry pattern to manage shared information across the game and framework. This provides a simple way of mapping a game resource (e.g. Entity Type, Item, System) to an ID and sending it over the network. This has the downside that either the registries must be constructed in the same order, or the registry mappings must be synced once to ensure they are in alignment. For this project, the registries are constructed in a consistent order and rely upon binary compatibility.

4.2.3 Defining Game Systems

To enable systems to be executed on either the client or server, they were designed to be invoked independently of one another. Systems could be as simple as plain functions, but given the object oriented nature of C#, systems are represented by an interface which defines how systems are invoked. (See Listing 4.1). The game calls `Update` on each system which is enabled locally every frame, providing it a reference to the game and the number of seconds since the last frame.

```

1 // Accepts a generic parameter for the game type, this is an artifact of keeping the ↔
   framework separate
2 public interface IGameSystem<in TGame> where TGame : AbstractHybridGame<TGame>
3 {
4     // Access to the system's identity, environment constraints and impact weight/ ↔
       score.
5     // More on this type shortly...
6     GameSystemType Type { get; }
7
8     // System update logic, with access to the current game instance (containing all ↔
       entities and other state)
9     void Update(TGame game, float deltaTime);
10 }

```

Listing 4.1: Game System Interface

Each system has a `GameSystemType`, which are collected in a registry. This serves as a way to identify each system, and store a list of which system types are enabled on each side. It also makes it simple to send an enable event, as the registry ID for the system type can be used. These types also provide a couple of properties used for system balancing, detailed in Section 4.2.4.

Systems must not store any state outside the game, such as in its class. This ensures that no data is lost if the remote server becomes unreachable before it could send any data back to the client. It also makes swapping systems between sides inexpensive because no extra state needs to be sent.

4.2.4 Dynamic System Balancing

The game will evaluate its performance metrics every second to determine whether the game is underperforming. If this is the case, the game will attempt to offload the highest impact system running locally however, if it detects the game is performing well, it will attempt to bring the lowest impact remote system local to make use of the spare local compute. This algorithm is shown in Listing 4.2.

Which system is offloaded first depends on the currently active systems, and their system type properties (`GameSystemType`) which specifies an environment constraint and an impact score. The environment constraint can be one of: `LocalOnly`, `HybridAny` or `PreferLocal`. The system will keep any `LocalOnly` systems local, regardless of current local performance. Then, `PreferLocal` systems will only be offloaded if the game performance is still suboptimal after offloading all `HybridAny` systems. The impact score is a weighting set by a developer to provide an artificial weighting to the order of the systems (within these environment constraint brackets), and is used to

make higher impact systems candidates for offloading before low impact systems.

```
1 // Executed every second on the client
2 if (_systemRunTimeThisSecond > 0.1f || _updatesThisSecond < Application. ←
    targetFrameRate / 2)
3 {
4     // Attempts to offload the highest-impact system locally.
5     TryOffloadSystem();
6 }
7 else if (_systemRunTimeThisSecond < 0.1f && _updatesThisSecond >= Application. ←
    targetFrameRate / 4 * 3)
8 {
9     // Attempts to offload the lowest-impact system remotely.
10    TryOnloadSystem();
11 }
```

Listing 4.2: Dynamic Offloading Logic

4.2.5 The Event Bus

It became clear in the early stages of development that manually handling network sync can lead to the potential for the developer to forget to synchronise data. It also tends to result in code duplication, having one place in the code where you change data locally and send the network packet, and then another place which receives and handles this packet.

Considering these factors, the framework was rearchitected around a networked event bus. This enabled game state changes to be handled in a centralised and consistent manner - to update the game state, an event must be posted to the bus, and the bus will forward this event over the network to the remote peer.

To facilitate the network transmission, all events derive from the `AbstractEvent` class (shown in Listing 4.3). This class exposes a network channel number and delivery method, used to determine how to send the event. More complex events can create a custom packet to encapsulate data differently, for example using a registry to acquire an ID.

```
1 public abstract class AbstractEvent {
2     // Channel and Delivery Method should be the same across all packets of the same ←
    type.
3     public abstract byte Channel { get; }
4     public abstract DeliveryMethod NetworkDeliveryMethod { get; }
5 }
```

Listing 4.3: AbstractEvent Class

During testing, a high rate of events resulted in performance problems. As such, a new method on the event bus was added to fire up to 32 events simultaneously (keeping the packet size within library limits). This bundles everything into a single packet, reducing the number of individual packets sent, and noticeably improved performance.

4.3 Test Game

4.3.1 Game Systems

The test game features eight systems which form the core gameplay loop: Physics, Damage, Weapon, Director, Distanced Follower, Dumb Follower, Experience, and Particles. This section will only discuss systems which are of specific interest and demonstrate how the framework handles gameplay behaviours.

Physics System

The physics system is responsible for handling entity movements and preventing overlapping. Each entity has a position, a movement vector, a movement speed, and a collision layer which are used by the physics system.

The physics system iterates over every entity and moves it along its movement vector while ensuring that other entities in its layer do not intersect. If an entity intersects, it moves the entity back out of the collision.

To ensure there are no unexpected behaviours as a result of entities moving during the physics system stepping, it will collect all movements and apply them after the entire set of entities has been resolved.

Damage System

The damage system handles damageable entities colliding with damage sources and applies the damage and invincibility frames. To prevent applying more damage than expected, all interactions are stored until all entities have been checked.

Invincibility frames were an interesting part of this system. Typically they can be implemented using a counter decrementing each frame but it would be wasteful to send events to do this every frame. Instead invincibility is handled by setting an `InvincibleUntil` field to the current game time plus the duration of the invincibility frames. Entities can then be tested for invincibility by testing the current game time against this field.

Weapon System

To simplify the process of creating enemies, an item system was designed to be shared between enemies and the player. This enables the re-use of weapons across the game and vastly reduces the amount of code necessary to implement special behaviours. Most enemies are not given weapons, however Wizards were given the ‘Magic Staff’ to demonstrate this capability. All items are defined with ”components” which detail how they operate and scale (all inventories can have multiple stacks of any item to increase its strength). An example of the magic staff’s definition is shown in Listing 4.4. Other item components also exist for increasing the player’s statistics, but these were not implemented for enemies as they were not used in the test game.

```
1 public static Item MagicStaff = Item.Create()
2     // The particle to spawn when fired
3     .AddComponent(ItemComponents.WeaponParticle, EntityTypes.MagicParticle)
4     // The time (in seconds) between firing each shot
5     .AddComponent(ItemComponents.WeaponDelay, (stackAmount) => Mathf.Max(4f * Mathf. ←
6         Exp(-0.4f * stackAmount), 0.1f))
7     // The number of shots to be fired each time
8     .AddComponent(ItemComponents.ParticleCount, (stackAmount) => Mathf.Min(( ←
9         stackAmount * 2) - 1, 16))
10    // Damage multiplier, applied to each fired shot
11    .AddComponent(ItemComponents.DamageMultiplier, (stackAmount) => Mathf.Min(Mathf. ←
12        Exp(0.2f * stackAmount), 20))
13    .Build();
```

Listing 4.4: Example weapon item definition

The weapon system is responsible for iterating over every entity with an inventory and firing any weapon items in the inventory once its delay has been reached, with a minor offset unique to each entity to make everything feel more natural. The offset is computed with a random number generator, seeded with the entity ID ensuring that both the client and server always generates the same offset for identical entity instances.

Director System

The “director” system is responsible for the spawning of waves of enemies during the game. It uses the in-game timer to determine the level difficulty, ensuring that as time progresses the game becomes more difficult. Using this level, it determines how frequently to spawn enemies and how many enemies can be spawned (in a measure of ‘credits’). Each enemy type has a level constraint (min/max level), a weight (how frequently it is picked at random) and a cost. Enemies will continue to be randomly picked until all of the director’s credits have been spent. By relying on the game timer

for scaling the enemy spawning, this system can be moved between the client and server at any time with no gap in enemy waves.

Particle System

The particle system is a small system that handles particle lifetimes and movement patterns. Lifetimes work similarly to damage invincibility, they are allocated a time upon spawning at which they must be removed. In addition to this, it also handles setting a particles initial direction based on the particle type. Some particles are set into a completely random direction, while others will target a random enemy (or player if fired by an enemy).

4.3.2 Link Between Virtual Level and Unity Scene

Due to the design of the framework, the game "world" (its entities in this case) are stored in a data structure that is separate from Unity's concept of a Scene. As such, it was necessary to build a bridge between the "virtual" world and the Unity scene in order to display everything on the screen. This was handled by the `ClientLevelManager`, a `MonoBehaviour` that was already responsible for creating and holding the client level. The client level was given a reference back to this behaviour, and enabled it to trigger entity spawns, positional changes and event particle effects whenever the relevant events were posted.

For example, in the `ClientLevel` class, `OnEntitySpawn` was overridden and would pass the event details onto the `ClientLevelManager` (Listing 4.5).

```
1 protected override void OnEntitySpawn(EntitySpawnEvent entitySpawnEvent) {  
2     base.OnEntitySpawn(entitySpawnEvent);  
3     _clientLevelManager?.SpawnEntity(entitySpawnEvent.Entity);  
4 }
```

Listing 4.5: `OnEntitySpawn` override on `ClientLevel`, passing the information to the Client Level Manager.

The `ClientLevelManager` will then lookup the entity's type against a map of prefabs and instantiate the prefab at the same location as the entity in the "virtual" level. These game objects are tracked in a map to the entity GUID, so whenever the entity moved it would be replicated within the Unity scene. This separation of game logic and presentation was not the most ergonomic pattern and could have been avoided with the use of `MonoGame`, however Unity was valuable for its battle readiness and platform support.

4.4 Automated Performance Benchmark

To measure the performance of the game, three benchmark modes were added. These have different settings to help evaluate the framework to identify its strengths and weaknesses. The three modes are described in Table 4.1, all benchmarks are run for 90 seconds.

Benchmark	System Execution Mode	Purpose
Standard	Systems will be dynamically balanced between the local and remote resource in the same manner as the standard game.	To evaluate whether the dynamic balancing of systems is beneficial to the games performance.
Local Only	No remote server involved, all systems executed locally.	Acts as a control test for a traditional local-only game.
Remote Only	Only runs systems remotely, client only handling presentation.	Acts as a worst-case scenario, where the local system is incapable of running any of the games systems.

Table 4.1: The three different benchmark types and their purpose.

The benchmark was implemented within the game to make testing across multiple devices as consistent and effortless as possible, making sure all evaluations are fair. During each benchmark, the game’s performance state is queried every second for both the client and server (separately) - it captures the following metrics:

- Entity count
- Frames this second
- Events this second
- System count (number of systems running on this side)
- System run time (the amount of this second spent executing systems)
- Remote peer ping (the network latency between client & remote in ms)

Network statistics (e.g. bytes sent/received, latency) were also captured by the game, however these were not used during performance evaluation as they do not provide any useful context.

These metrics will be used during evaluation to determine how well the game performs, and evaluate the framework's design against the original requirements.

Chapter 5

Evaluation & Testing

5.1 Testing Methodology

To test the functionality of the game and the framework, the game was tested actively during development. After implementing major features (both in the framework and the game), the game would be tested and checked through the use of debug log messages to ensure that features of the framework were acting as expected. To ensure that events were being fired and handled correctly, the debugger was used - watching for specific events to ensure everything was being broadcast as expected. While gameplay was not a key focus of the project, the gameplay was also evaluated to ensure that it worked as expected and served as an effective demonstration of the framework.

To evaluate the performance of the system, the three automated benchmarks were run three times on two different mobile devices - one representative of a low-end device and the other representing high-end (See Table 5.1). This was done to ensure that the metrics gathered from each benchmark were consistent and correct. After each test, the metrics export (in CSV format) was stored in its relevant test subfolder in the project repository for evaluation. Each file was imported into a Jupyter Notebook and the data was visualised for evaluation and the evaluation of this data is presented in Section 5.3.

Finally, Section 5.4 will discuss a number of pitfalls and difficulties during development and discuss how they potentially impact the usability of the framework.

Device	CPU	RAM	GPU	Network
Development Machine and Remote Server	Intel Core i7-12700k	64 GB	AMD Radeon RX 7900 XTX	Connected to local Wi-Fi and directly to the Android VM via Hypervisor.
Android Virtual Machine	1 Core	1 GB	Forced Software Acceleration	Connected to server via hypervisor.
iPhone 16 Pro Max	6 Cores (2 Performance, 4 Efficiency)	8 GB	6 Cores	Connected to server via local Wi-Fi

Table 5.1: Devices Used for Performance Evaluation

5.2 Functionality Testing

To test the functionality of the framework, the game was designed to exercise all framework functionality, ranging from game state management and synchronisation to offloaded gameplay systems. The game was tested iteratively: after the introduction of new functionality, the game was tested to verify it. This was performed within the Unity editor where debugging tools were readily available to diagnose and trace behaviours.

Functional testing demonstrated that game state synchronisation worked as expected, confirmed by debug messages showing matching entity counts on both sides. It also demonstrated that systems were correctly offloaded by stepping through both the client and server loops to inspect what was happening at runtime. Finally, the remote only benchmark demonstrated that network sync was working as expected, as the game played correctly despite all systems being executed on the server.

All major bugs in both the game and the framework were addressed immediately upon discovery, and lower priority issues were addressed depending on their severity. All known framework bugs were addressed before running performance tests, while gameplay bugs were lower priority due to their limited effect on the project's outcome. For example, there was a physics bug where two entities spawning on top of each other causes extreme and random movements, which was not addressed due to its low priority. Overall, functionality testing confirmed that the framework operates as required for this project.

5.3 Performance Testing

To establish a baseline for a ‘standard’ game, the local only benchmark was used to investigate the game performance when no networking is involved. Figure 5.1 demonstrates that the iPhone is capable of running the game locally with no difficulty whereas the Android virtual machine suffers from worsening frame drops after approximately 35 seconds.

Benchmark	Device	Average FPS	1% Lows
Standard	iPhone	60.01	60.00
Standard	Android VM	47.43	30.67
Remote Only	iPhone	60.00	60.00
Remote Only	Android VM	49.49	38.33
Local Only	iPhone	60.14	59.33
Local Only	Android VM	51.91	34.33

Table 5.2: Frame rate averages and 1% lows for all benchmarks

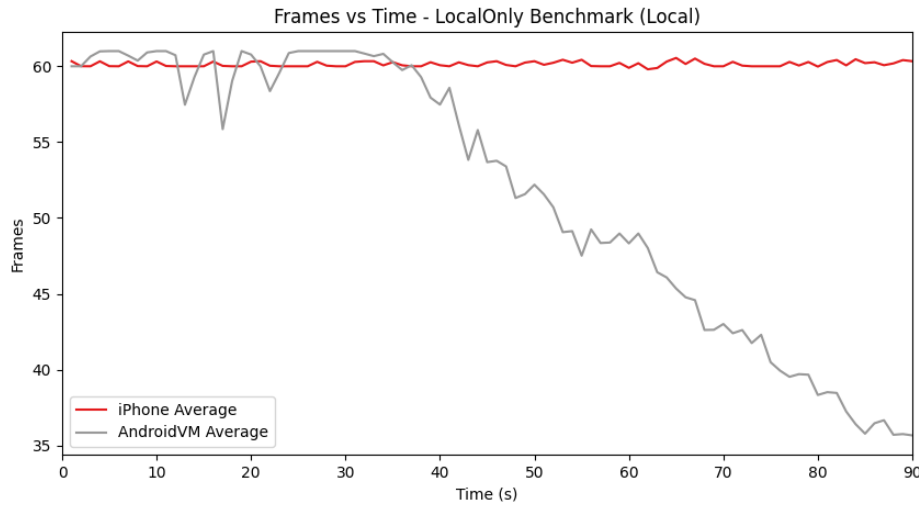


Figure 5.1: Average frame rate over time on the client during the local only benchmark for both devices.

When comparing the average frame rates and 1% lows in Table 5.2, it is clear that the iPhone performs consistently across all benchmarks when comparing frame rates. However looking at the Android results, performance degrades when it begins communicating over the network. The remote only benchmark does provide better 1% low frame rates, but this is unlikely to be noticeable by the player. Overall, these

results indicate that the networking overhead from the event bus is reducing overall performance, rather than increasing it. This is reinforced when comparing Figures 5.2 and 5.3, which shows that during the standard benchmark, performance is degraded immediately whereas the local benchmark starts to drop off around the 35 seconds mark.

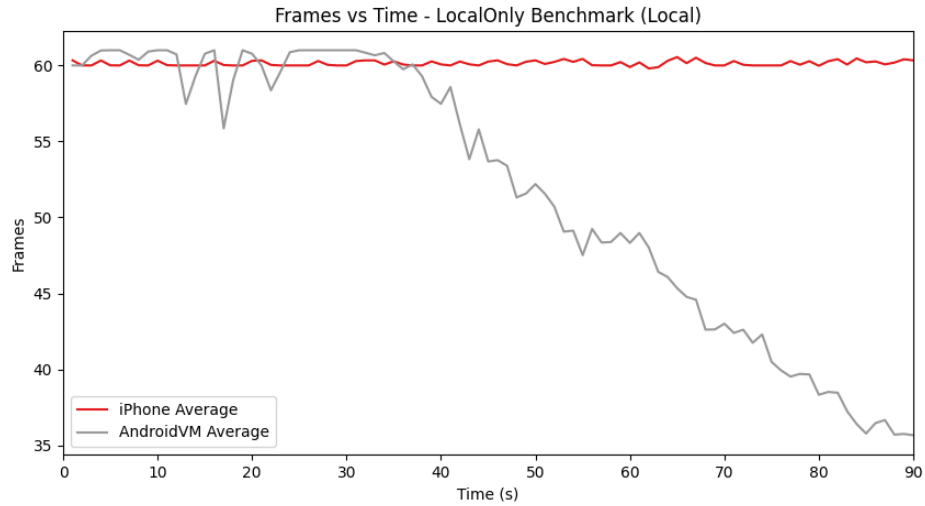


Figure 5.2: Average frame rate over time on the client during the local only benchmark for both devices.

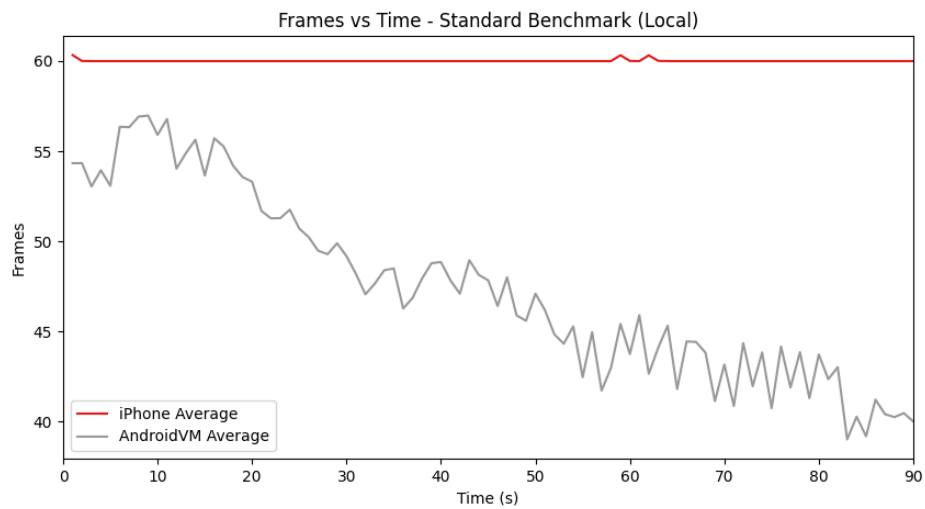


Figure 5.3: Average frame rate over time on the client during the standard benchmark for both devices.

To ensure that both the local and remote sides are completely in sync, the Pearson

correlation coefficient was calculated for the Event counts on the local and remote sides of the game to ensure that both sides were always handling the same number of events. As seen in Table 5.3, all of the benchmarks have highly correlated event counts, there is a minor drop for the standard iPhone benchmark but this is likely due to the use of Wi-Fi network conditions causing some unreliable events to be dropped.

Benchmark	Device	Correlation Coefficient
Remote Only	iPhone	0.996
Remote Only	Android VM	0.998
Standard	iPhone	0.989
Standard	Android VM	0.938

Table 5.3: Pearson correlation coefficients for event counts across the local and remote sides of the benchmarks.

Another observation that can be made from Figure 5.4 is that the system balancing logic has a flaw, once the games performance reaches a certain point it will on-load and consequently offload the most impactful system on repeat. Both the local and remote frame times fluctuate between high and low every frame, reflecting a system being passed between the two sides repeatedly. This is because once the system is offloaded, the game performance improves enough to trigger on-loading. This indicates that the balancing system should be equipped to deal with this scenario better.

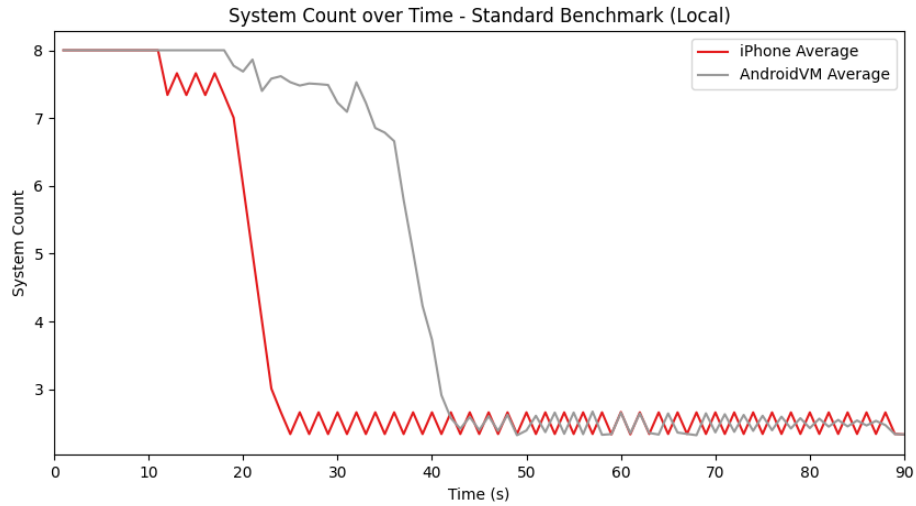


Figure 5.4: System count over time for the standard benchmark on both devices.

Another interesting finding is that the iPhone off-loads systems earlier than the Android VM. This may be explained by observing the system run time on the local only benchmark (Figure 5.5). It shows an abrupt drop in system run time around the 50-60 seconds mark, this is most likely due to the iPhone’s CPU scheduler moving the game from an efficiency core to a performance core. Figure 5.6 shows that the system runtime increased more slowly on the Android VM until the iPhone switched to its performance core.

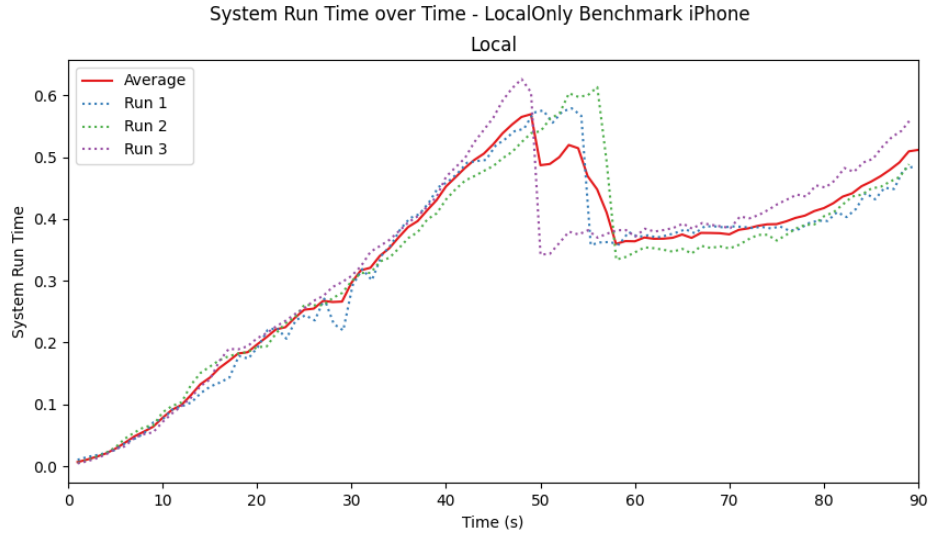


Figure 5.5: System run time drops at the 50-60 second mark in all three runs while running locally on the iPhone.

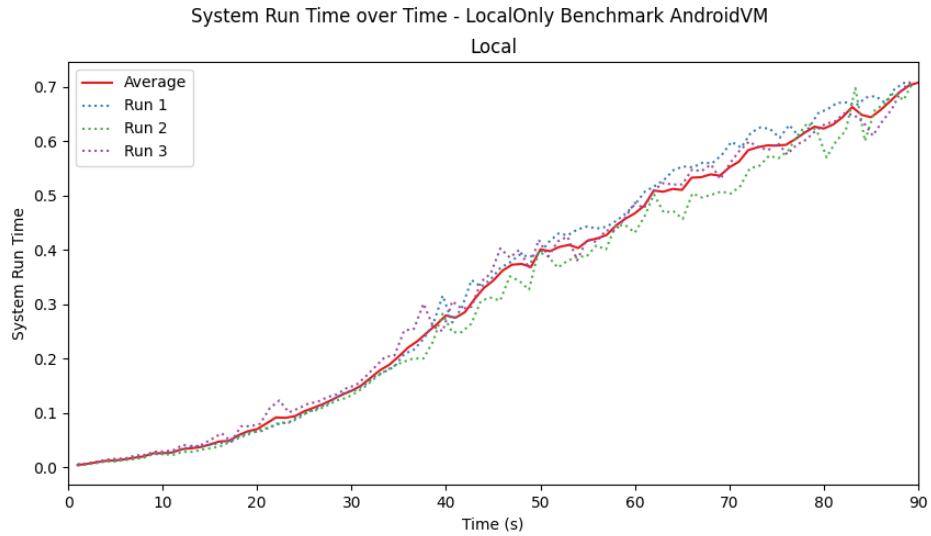


Figure 5.6: System run time over time during the local only benchmark on the Android VM.

It is clear from these performance metrics that the system is functional and is capable of reducing the computational load on the iPhone. However, it is not capable of improving the gameplay experience for lower-end devices.

5.4 Framework Evaluation

5.4.1 Event Bus Pitfalls

The event bus approach to managing game state changes has a number of drawbacks. Firstly, firing events within the handler for another event should be done with care. If this occurs on both sides, this could result in the firing of multiple events unintentionally. This has the potential to have significant performance implications, depending on the frequency of these unintended events.

Secondly, special care should be taken when mutating the game's state in an event handler. While this is the purpose of an event, all changes should be deterministic to avoid any one peer falling out of sync from another.

Finally, the event bus experiences a high amount of traffic during the game, and consequently generates a high amount of network traffic. To help ease this, the event bus could be redesigned to group similar packets together for bundled dispatch each frame - or even send all events from the current frame at once to reduce the number of individual packets in transit.

5.4.2 Packets Stuck in Network Queues

Another challenge faced during the implementation was packets getting stuck in the reliable channel queues. This is because position and movement vector change events were being sent at a significantly high rate, and these were all being sent ordered and reliably. Eventually there was so much traffic that LiteNetLib's channel queue was filling up because more events were being fired than could reasonably be buffered and checked for arrival status. To remedy this, these events were given their own network channels and switch to unreliable mode - removing the need to queue the packets, but at the cost of the potential for data loss (and thus de-sync). This was necessary to ensure that the server didn't fall completely out of sync while the client was backing up with packets.

This indicates the need for extreme caution when sending high-speed events through the event bus, and perhaps the periodic re-sending of these unreliable events (perhaps with an independent system confined to the client which picks random entities that have been stationary for some time and re-sends their position).

Chapter 6

Conclusions and Future Work

6.1 Conclusions

The framework’s development process resulted in a number of important lessons learned, and generated a significant amount of data to help evaluate its effectiveness. Most importantly, it demonstrates that the networked event bus was capable of acting as a synchronisation system between the client and server despite its need for further optimisation. It also demonstrated that it is possible to move individual parts of the gameplay loop onto a remote resource and continue to have a playable game.

Looking back on the project objectives, the following conclusions can be drawn:

1. This project does not prove that hybrid computation for video games is an effective cloud gaming strategy, but nor does it disprove it. The strategy explored in this project was not effective for the low-end device, however showed some promise for the high-end device. More work would be required to conclusively determine whether hybrid computation can be effective.
2. This project does demonstrate a capability to make efficient use of local resources, demonstrating that the game will move systems locally at times it does not affect the gameplay experience.
3. This project does not enable lower-end devices to play currently inaccessible titles. As shown during the performance evaluation, the networking overhead of this solution outweighs the benefit of moving game logic onto the cloud.
4. Drawing upon the system runtime, it is clear that the game is performing less work than it would running the game fully locally. Unfortunately, this is not enough information to prove whether this reduces the energy consumption of

games to extend playtime. More testing would be required to demonstrate this, such as real-world battery benchmarking.

5. This solution does protect game ownership rights, the developed game is fully playable without the remote server.

6.2 Personal Reflection

I am fairly happy with the finished test game and framework. While it does not perform as I had hoped, I also admit that I suspected that networking overhead would nullify any performance benefit in a game this simple. I am optimistic that hybrid cloud gaming is a path towards bringing video games to lower tier devices, however I think the strategies to offload graphics continue to show more promise. I think this is plainly because most games do not have gameplay logic that is more intensive than their rendering logic, except for the occasional game which is logic intensive, for example large simulation games.

If I was to start this project again, I would not have sent each individual event as its own network packet, but gathered events in bulk and sent them once enough had accumulated. This would have helped to reduce the network traffic issues encountered during this project. This was the primary pain point during implementation, and is the main thing I wish I had approached differently as I believe this would have helped to improve the performance results.

Ultimately I believe that the strategies proposed by this project have significant potential and that they could be adapted to better suit low-end devices and to fully demonstrate that hybrid computation is a valuable strategy to improving access to video games.

6.3 Future Work

Future works could investigate strategies for a more efficient networked event bus. Instead of sending events over the network immediately, they could be gathered and sent at the end of each frame. This would potentially reduce the networking overhead, and could be used in conjunction with other strategies such as logic to drop unnecessary events to reduce traffic further (e.g. dropping a movement event for an entity that has another later this frame).

Other works could investigate better strategies for offloading systems to avoid the jagged effect of systems moving between peers every other second. This could involve collecting runtime statistics of real-world performance of each system and using that

to determine which systems are most impactful, and using this information to avoid moving a high-impact system local again.

Data from the server during benchmarking indicates that the frame/update rate of the server drops significantly when running a single remote game, this is not acceptable for a properly scalable solution. Future works could investigate ways to optimise the remote game, perhaps reducing the update rate and introducing network interpolation to reduce the amount of work being done, as well as using parallelisation to separate different play sessions on their own threads.

Chapter 7

Bibliography

- Balasubramanian, K. (Sept. 2022). *The Decline of Physical Games and The Rise of Digital Distribution*. URL: <https://www.gameopedia.com/decline-of-physical-games/> (visited on 23/10/2024).
- Beets, K. (Jan. 2021). *Mobile Game Development: Differences, Challenges, New Solutions*. URL: <https://80.lv/articles/mobile-game-development-differences-challenges-new-solutions/> (visited on 22/10/2024).
- Cable.co.uk (2024). *Worldwide Broadband Speed League 2024*. URL: <https://www.cable.co.uk/broadband/speed/worldwide-speed-league/#historical> (visited on 22/10/2024).
- Chan, K. L. et al. (June 2017). ‘A Hybrid-Streaming Method for Cloud Gaming: To Improve the Graphics Quality delivered on Highly Accessible Game Contents’. In: *International Journal of Serious Games* 4 (2). ISSN: 2384-8766. DOI: [10.17083/IJSG.V4I2.163](https://doi.org/10.17083/IJSG.V4I2.163). URL: <https://journal.seriousgamesociety.org/index.php/IJSG/article/view/163>.
- Chan, S. (July 2017). *Mobile game revenue finally surpasses PC and consoles — VentureBeat*. URL: <https://venturebeat.com/games/mobile-game-revenue-finally-surpasses-pc-and-consoles/> (visited on 22/10/2024).
- Chen, K. T. et al. (2011). ‘Measuring the latency of cloud gaming systems’. In: *MM’11 - Proceedings of the 2011 ACM Multimedia Conference and Co-Located Workshops*, pp. 1269–1272. DOI: [10.1145/2072298.2071991](https://doi.org/10.1145/2072298.2071991). URL: <https://dl.acm.org/doi/10.1145/2072298.2071991>.
- Choi, Y. et al. (Oct. 2022). ‘Optimizing Energy Consumption of Mobile Games’. In: *IEEE Transactions on Mobile Computing* 21 (10), pp. 3744–3756. ISSN: 15580660. DOI: [10.1109/TMC.2021.3058381](https://doi.org/10.1109/TMC.2021.3058381).

- Choy, S. et al. (2012). ‘Edgecloud: A new hybrid platform for on-demand gaming’. In: *School Comput. Sci., Univ. Waterloo, Waterloo, ON, USA, Tech. Rep. CS-2012-19*. URL: https://www.academia.edu/download/43531075/EdgeCloud_A_New_Hybrid_Platform_for_On-D20160308-31760-j1ay9e.pdf.
- (Sept. 2014). ‘A hybrid edge-cloud architecture for reducing on-demand gaming latency’. In: *Multimedia Systems* 20 (5), pp. 503–519. ISSN: 14321882. DOI: [10.1007/S00530-014-0367-Z](https://doi.org/10.1007/S00530-014-0367-Z). URL: <https://link.springer.com/article/10.1007/s00530-014-0367-z>.
- Cloudflare (2024). *What is latency? — How to fix latency — Cloudflare*. URL: <https://www.cloudflare.com/en-gb/learning/performance/glossary/what-is-latency/> (visited on 23/10/2024).
- Domenico, A. D. et al. (Dec. 2020). ‘A network analysis on cloud gaming: Stadia, GeForce Now and PSNow’. In: *Network* 1 (3), pp. 247–260. DOI: [10.3390/network1030015](https://doi.org/10.3390/network1030015). URL: <http://arxiv.org/abs/2012.06774> <http://dx.doi.org/10.3390/network1030015>.
- Edgegap (2022). *Edgegap - Latency Report*. URL: <https://edgegap.com/gaming/case-studies/latency-report> (visited on 23/10/2024).
- Eisert, P. and P. Fechteler (Oct. 2008). ‘Low delay streaming of computer graphics’. In: *Proceedings - International Conference on Image Processing, ICIP*, pp. 2704–2707. ISSN: 15224880. DOI: [10.1109/ICIP.2008.4712352](https://doi.org/10.1109/ICIP.2008.4712352).
- Gaikai (Mar. 2009). *Gaikai*. URL: <https://web.archive.org/web/20090321065857/http://www.gaikai.com/> (visited on 27/10/2024).
- Google (2024). *Stadia*. URL: <https://stadia.google.com/gg/> (visited on 27/10/2024).
- Halbhuber, D. et al. (Sept. 2023). ‘The Effects of Latency and In-Game Perspective on Player Performance and Game Experience’. In: *Proceedings of the ACM on Human-Computer Interaction* 7 (CHI PLAY), pp. 1308–1329. ISSN: 25730142. DOI: [10.1145/3611070](https://doi.org/10.1145/3611070). URL: <https://dl.acm.org/doi/10.1145/3611070>.
- Huang, C. Y. et al. (2013). ‘GamingAnywhere: An open cloud gaming system’. In: *Proceedings of the 4th ACM Multimedia Systems Conference, MMSys 2013*, pp. 36–47. DOI: [10.1145/2483977.2483981](https://doi.org/10.1145/2483977.2483981). URL: <https://dl.acm.org/doi/10.1145/2483977.2483981>.
- Koulaxidis, G. and S. Xinogalos (Mar. 2022). ‘Improving Mobile Game Performance with Basic Optimization Techniques in Unity’. In: *Modelling 2022, Vol. 3, Pages 201-223* 3 (2), pp. 201–223. ISSN: 2673-3951. DOI: [10.3390/MODELLING3020014](https://doi.org/10.3390/MODELLING3020014). URL: <https://www.mdpi.com/2673-3951/3/2/14/htm> <https://www.mdpi.com/2673-3951/3/2/14>.

- Lindzon, J. (Jan. 2024). *The gaming industry is aiming for subscribers. Will gamers play along?* - *BBC Worklife*. URL: <https://www.bbc.com/worklife/article/20240130-the-gaming-industry-is-aiming-for-subscribers-will-gamers-play-along> (visited on 22/10/2024).
- LizardByte (2024). *Overview - Sunshine documentation*. URL: <https://docs.lizardbyte.dev/projects/sunshine/en/latest/about/overview.html> (visited on 27/10/2024).
- Microsoft (2024). *Xbox Cloud Gaming (Beta)*. URL: <https://www.xbox.com/en-GB/cloud-gaming> (visited on 16/10/2024).
- Nan, X. et al. (Dec. 2017). 'Delay-Rate-Distortion Optimization for Cloud Gaming with Hybrid Streaming'. In: *IEEE Transactions on Circuits and Systems for Video Technology* 27 (12), pp. 2687–2701. ISSN: 10518215. DOI: [10.1109/TCSVT.2016.2595330](https://doi.org/10.1109/TCSVT.2016.2595330).
- Netlify (Oct. 2018). *Netlify Edge*. URL: <https://www.netlify.com/platform/core/edge/> (visited on 27/10/2024).
- NVIDIA (2024). *GeForce NOW — The Next Generation in Cloud Gaming*. URL: <https://www.nvidia.com/en-gb/geforce-now/> (visited on 16/10/2024).
- OnLive (July 2010). *Welcome to OnLive.com*. URL: <https://web.archive.org/web/20100722102034/http://www.onlive.com/> (visited on 27/10/2024).
- Orland, K. (Apr. 2015). *OnLive shuts down streaming games service, sells patents to Sony* - *Ars Technica*. URL: <https://arstechnica.com/gaming/2015/04/onlive-shuts-down-streaming-games-service-sells-patents-to-sony-embargoed-7pm-eastern/> (visited on 27/10/2024).
- Osborne, H. (Apr. 2023). *What's the problem with cloud gaming?* URL: <https://stlpartners.com/articles/consumer/whats-the-problem-with-cloud-gaming/> (visited on 16/10/2024).
- poncle (2022). *Vampire Survivors*. URL: https://store.steampowered.com/app/1794680/Vampire_Survivors/.
- Prakash, A. et al. (2016). 'Improving mobile gaming performance through cooperative CPU-GPU thermal management'. In: *Proceedings of the 53rd Annual Design Automation Conference*. DAC '16. Austin, Texas: Association for Computing Machinery. ISBN: 9781450342360. DOI: [10.1145/2897937.2898031](https://doi.org/10.1145/2897937.2898031). URL: <https://doi.org/10.1145/2897937.2898031>.
- Rees, L. (July 2022). *Edgegap: lag and latency are major problems for mobile gamers — PocketGamer.biz*. URL: <https://www.pocketgamer.biz/edgegap-lag-and-latency-are-major-problems-for-gamers/> (visited on 23/10/2024).
- Satyanarayanan, M. (Jan. 2017). 'The emergence of edge computing'. In: *Computer* 50 (1), pp. 30–39. ISSN: 00189162. DOI: [10.1109/MC.2017.9](https://doi.org/10.1109/MC.2017.9).

- Shea, R. et al. (2013). ‘Cloud gaming: Architecture and performance’. In: *IEEE Network* 27 (4), pp. 16–21. ISSN: 08908044. DOI: [10.1109/MNET.2013.6574660](https://doi.org/10.1109/MNET.2013.6574660).
- Sony (2024). *Getting started with PS Plus — All you need to know about membership plans, PS Plus on PC, streaming and more (UK)*. URL: <https://www.playstation.com/en-gb/ps-plus/getting-started/> (visited on 16/10/2024).
- Statcounter (2025). *Mobile Operating System Market Share Worldwide*. URL: <https://gs.statcounter.com/os-market-share/mobile/worldwide> (visited on 28/4/2025).
- Statistica (2024a). *Cloud Gaming - Worldwide — Statista Market Forecast*. URL: <https://www.statista.com/outlook/amo/media/games/cloud-gaming/worldwide> (visited on 11/10/2024).
- (2024b). *Mobile Games - Worldwide — Statista Market Forecast*. URL: <https://www.statista.com/outlook/dmo/digital-media/video-games/mobile-games/worldwide#revenue> (visited on 11/10/2024).
- Steam (2024). *Steam Support :: Steam Remote Play*. URL: <https://help.steampowered.com/en/faqs/view/0689-74B8-92AC-10F2> (visited on 26/10/2024).
- Tenomichi Ltd (2016). *StreamMyGame*. URL: <https://web.archive.org/web/20160621133420/http://streammygame.com/smg/modules.php?name=OurSoftware> (visited on 18/10/2024).
- The NPC Jarvis (June 2024). *Destiny 2 - Beware of Data Mining!* URL: <https://www.zleague.gg/theportal/destiny-2-beware-of-data-mining/> (visited on 23/10/2024).
- Unity (Dec. 2023). *Optimize Your Mobile Game Performance (Unity 2022 LTS) — Unity*. URL: <https://unity.com/resources/optimize-mobile-game-performance-unity-2022lts?isGated=false> (visited on 22/10/2024).
- Vercel (Aug. 2024). *Edge Network*. URL: <https://vercel.com/docs/edge-network/overview> (visited on 27/10/2024).

Appendix A

Project Log

Date	Milestones
March 2024	• Discussed a couple of project ideas with my supervisor, and decided to pursue this project on hybrid game compute.
September 2024	• Had our first group meeting to discuss our project ideas. • Started research to help refine the project idea.
October 2024	• Completed the project research and established a simple set of requirements to inform the design phase. • Submitted the literature review/project scope for marking.
November 2024	• Shifted from group meetings to individual meetings to support the design and implementation phases of the project.

December 2024	• Established some documentation on the initial project plan, comprised of a process diagram and short description of the system design. • Had a meeting with my supervisor to discuss my project design plan before the Christmas break.
January 2025	• Created the initial MonoGame project, and started laying foundations. • Decided to use LiteNetLib as the networking library for the project. • Had issues using MonoGame with iOS, switched to Unity as a result. • Swapped from sending individual packets for game events to a networked event bus to handle game synchronisation.
February 2025	• Added a composable item system and the weapon particle system. • Achieved network synchronisation.
March 2025	• Added system offloading and the dynamic logic to do so automatically. • Finished adding weapons to the game. • Designed and submitted the project poster for printing.

April 2025	• Added performance metric recording for benchmarks. • Added the main game menu and automated benchmarks. • Resolved some serious performance challenges, mainly by adding more network channels and sending busy events unreliably. • Added an experience system for levelling up. • Ran all benchmarks and performed evaluations. • Finalised project report. • Final gameplay-specific tuning for the demo, improving enemy and weapon values to optimise for fun.
------------	---

Appendix B

Source Code

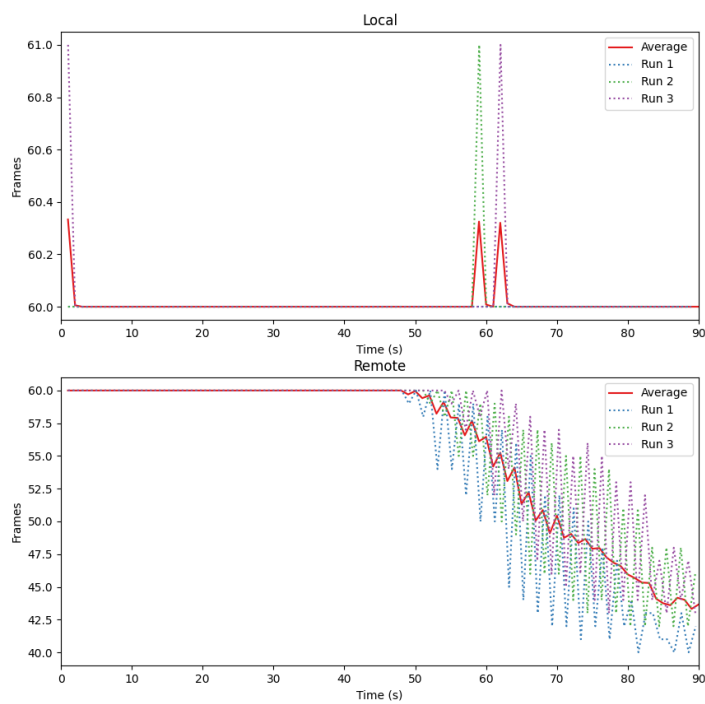
The source code for the Unity game, and the Jupyter Notebook used during evaluation is available on GitHub at: <https://github.com/Rover656/MSci-Project>.

Appendix C

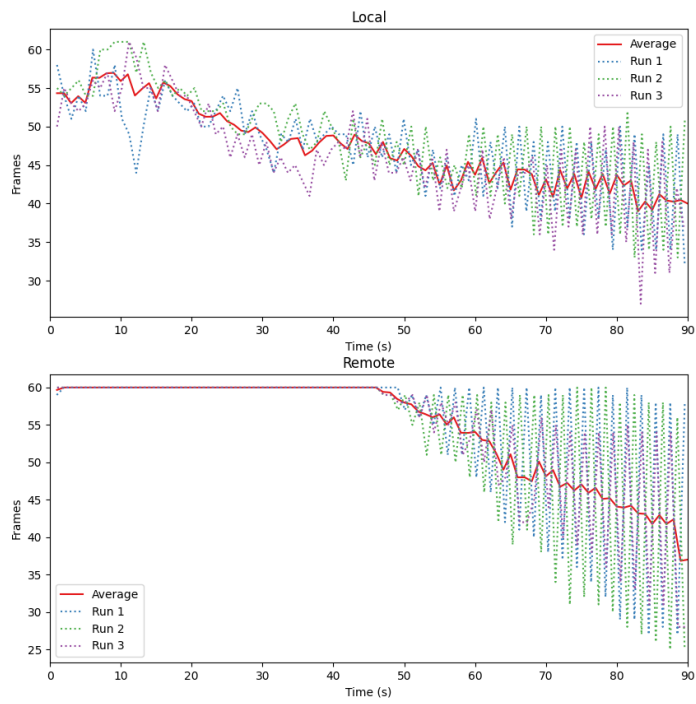
All Results

This contains all of the graphs generated during the evaluation of the framework.

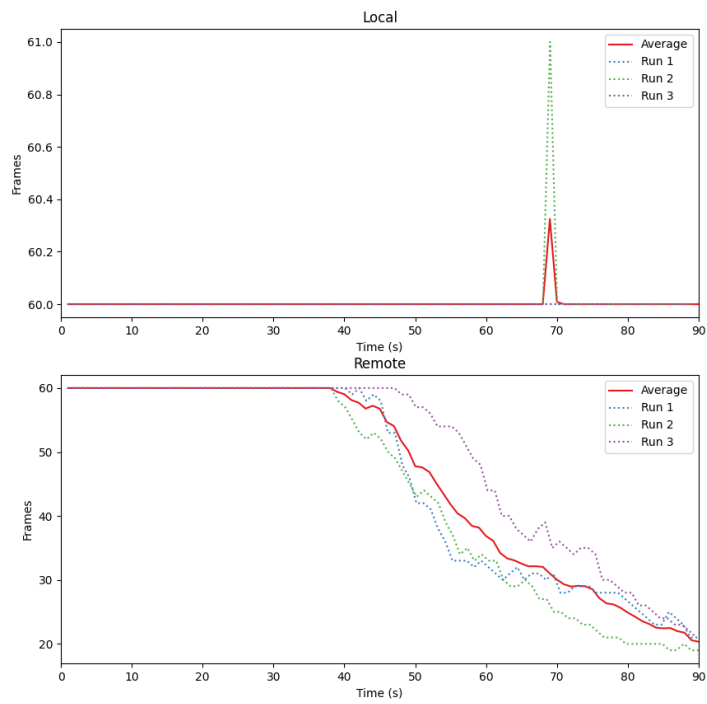
Frames vs Time - Standard Benchmark iPhone



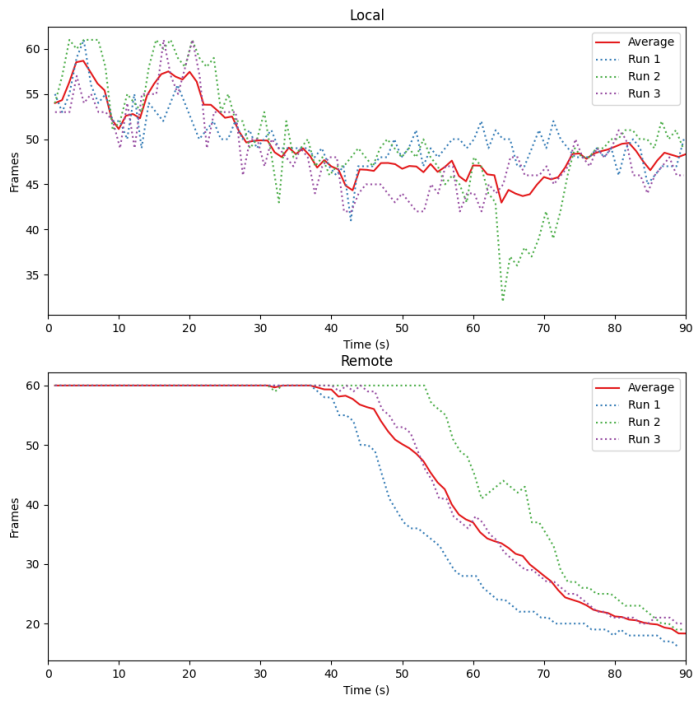
Frames vs Time - Standard Benchmark AndroidVM



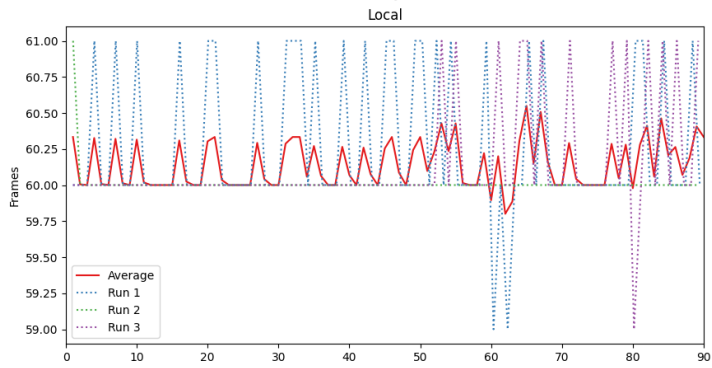
Frames vs Time - RemoteOnly Benchmark iPhone



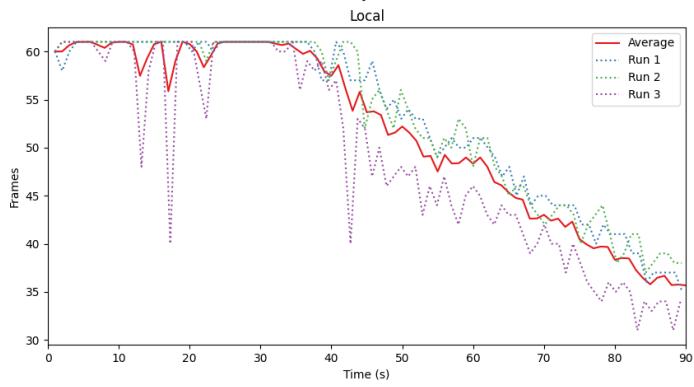
Frames vs Time - RemoteOnly Benchmark AndroidVM

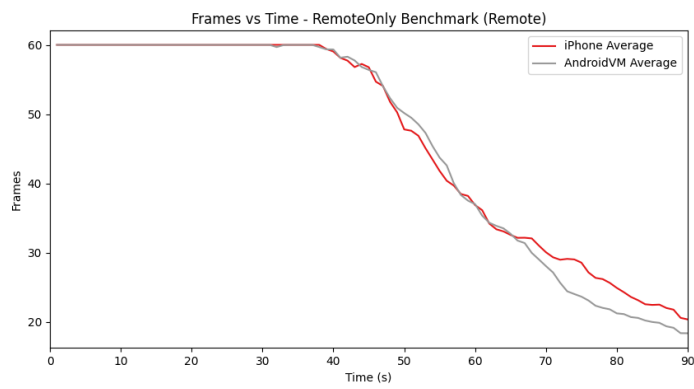
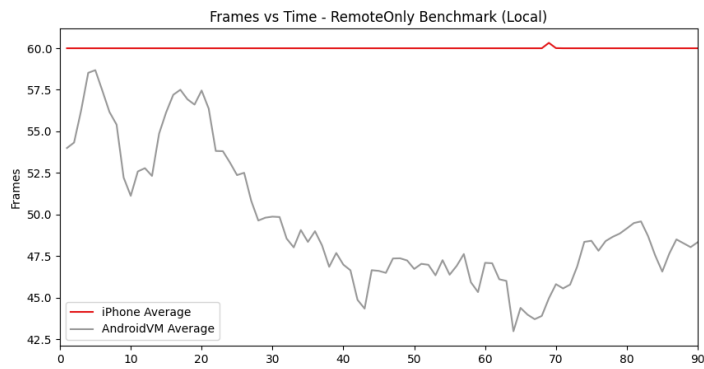
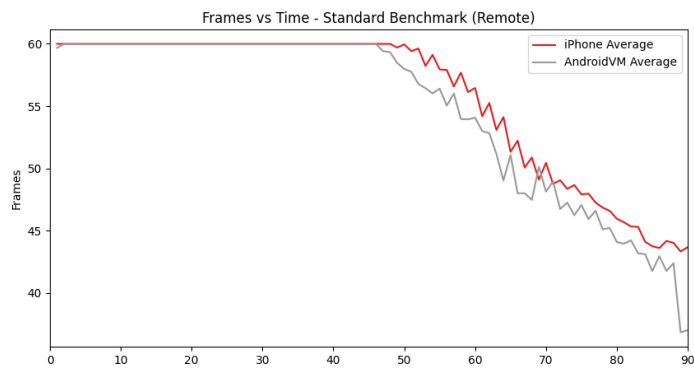
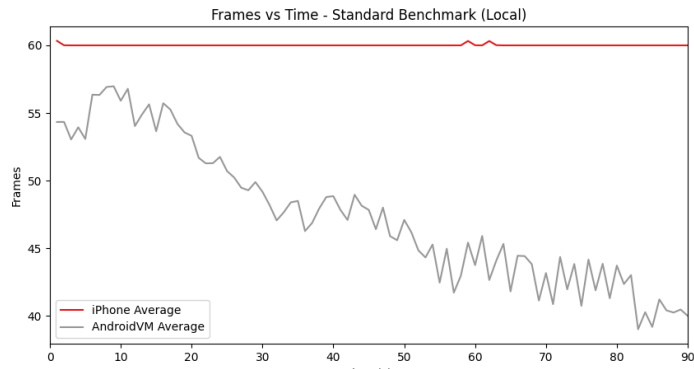


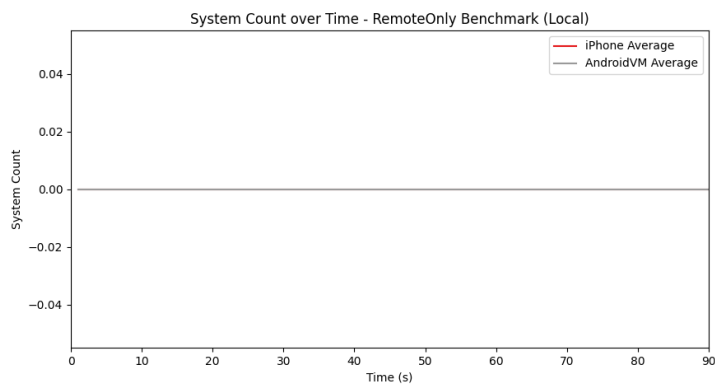
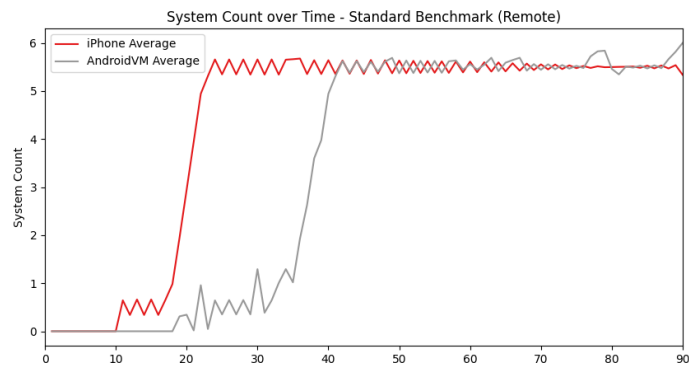
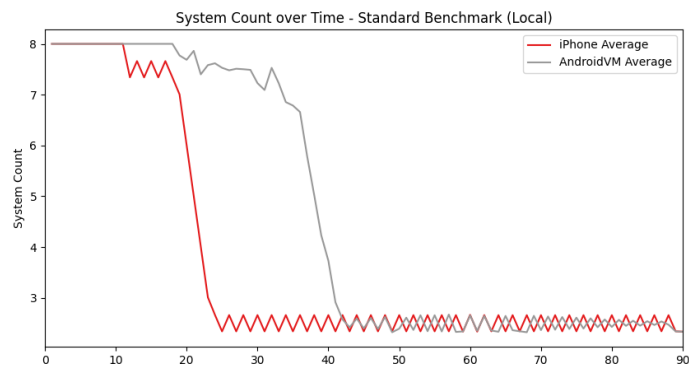
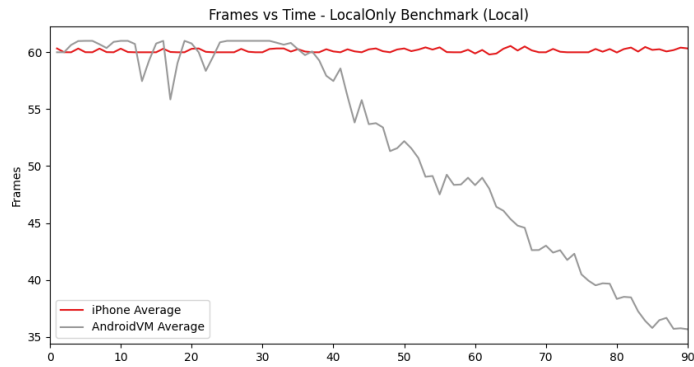
Frames vs Time - LocalOnly Benchmark iPhone

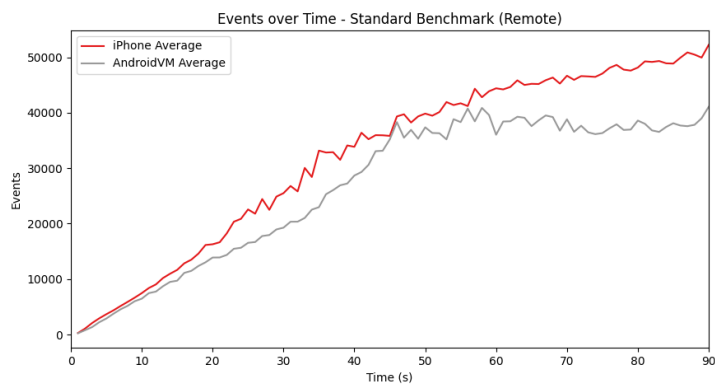
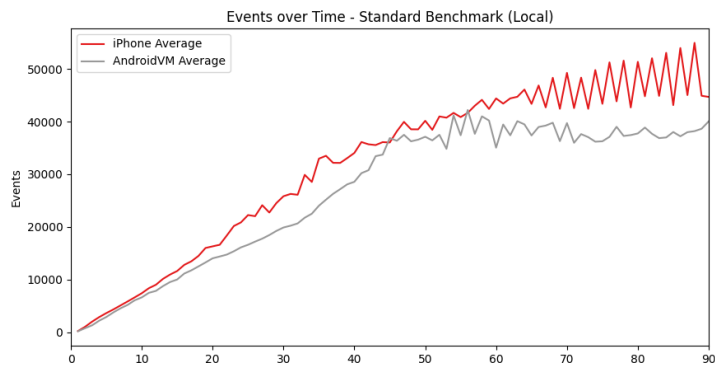
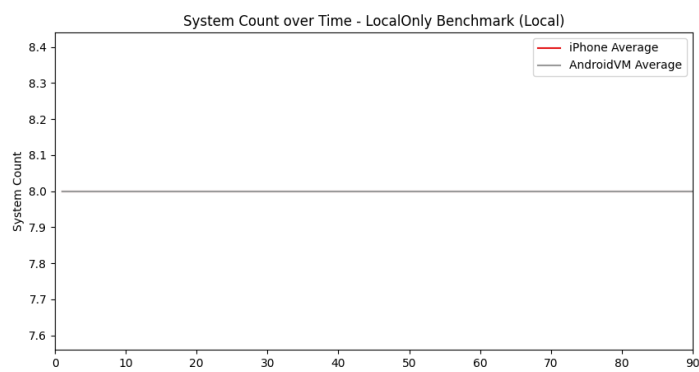
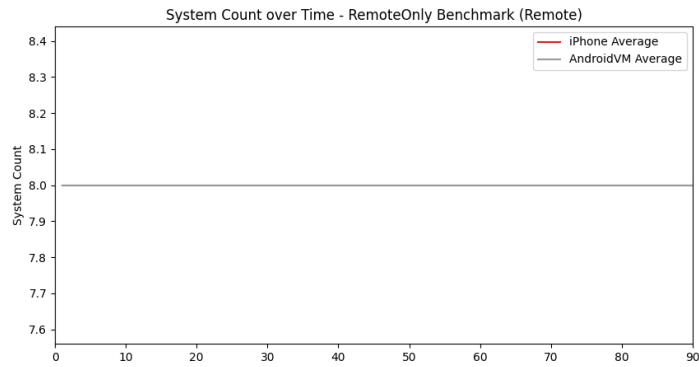


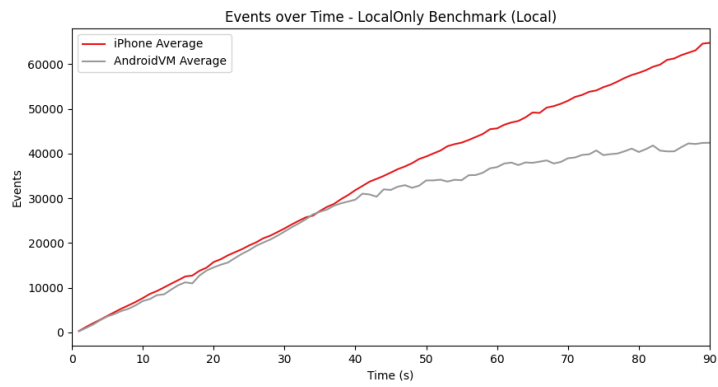
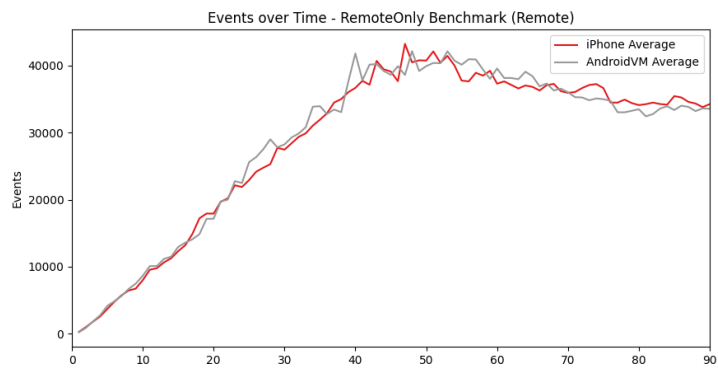
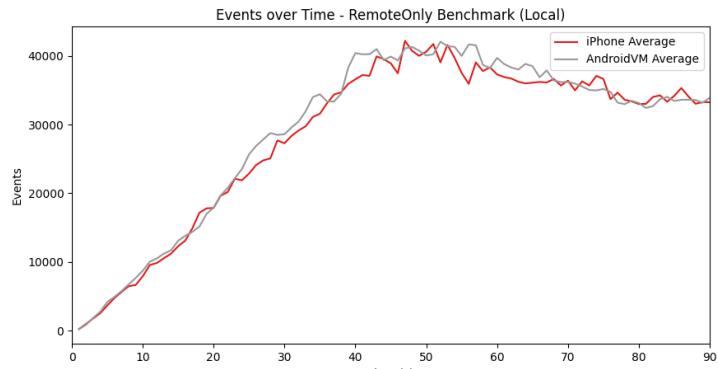
Frames vs Time - LocalOnly Benchmark AndroidVM



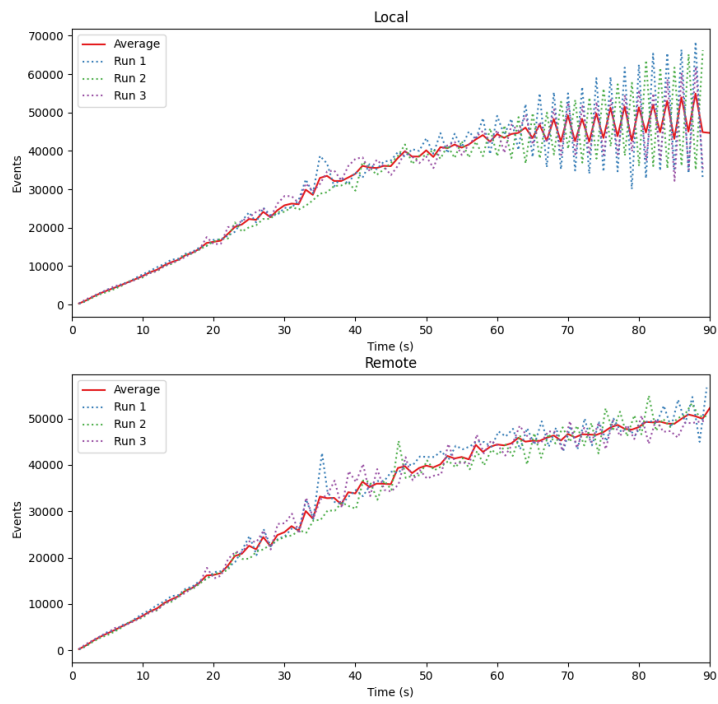




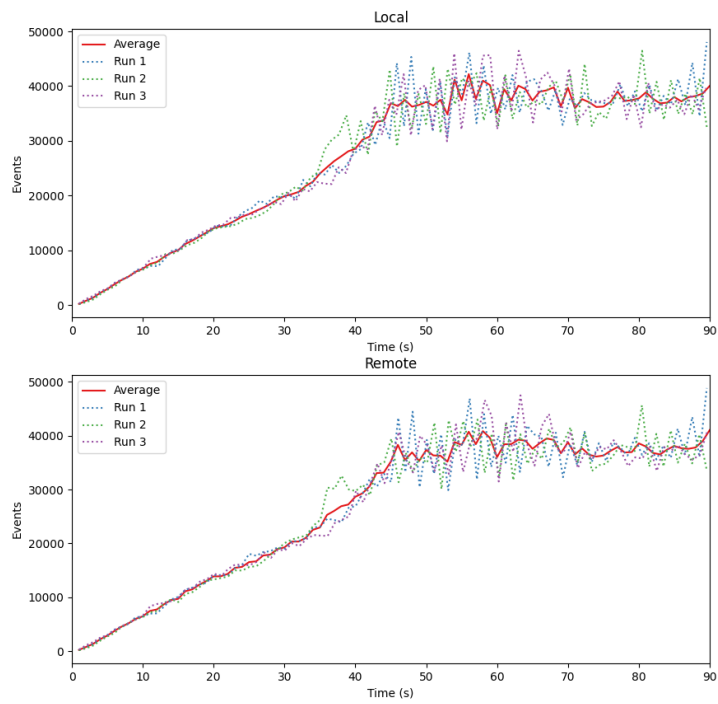




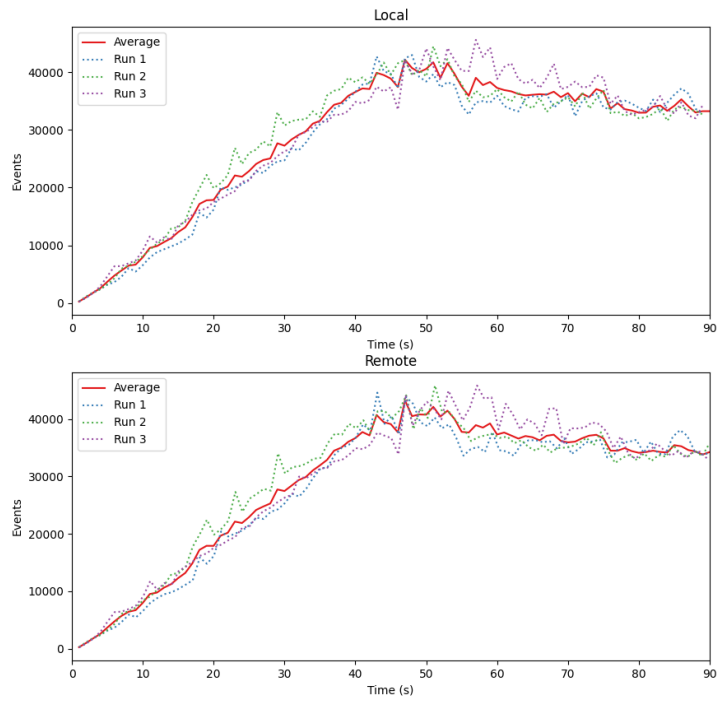
Events over Time - Standard Benchmark iPhone



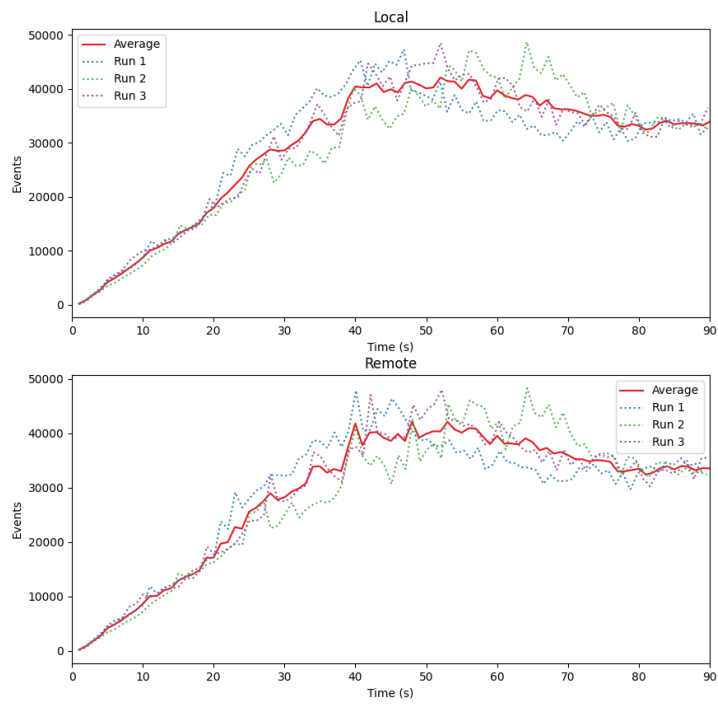
Events over Time - Standard Benchmark AndroidVM

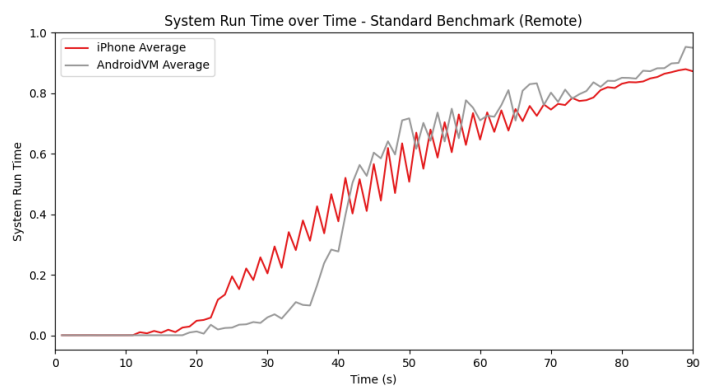
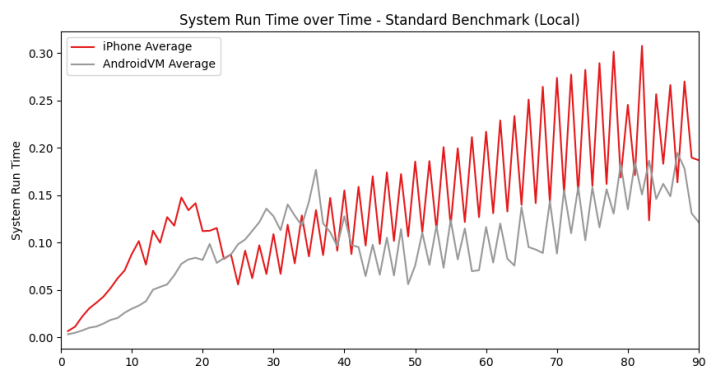
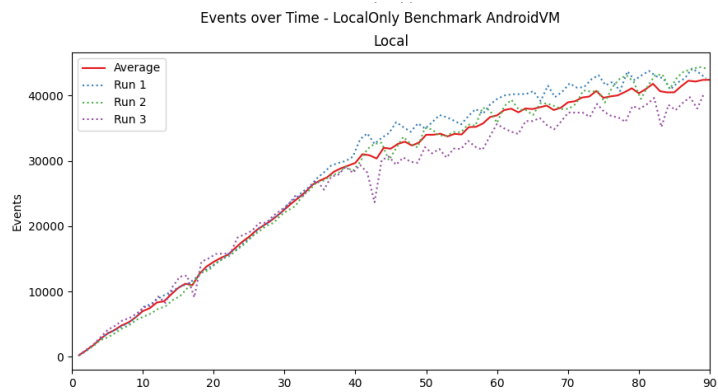
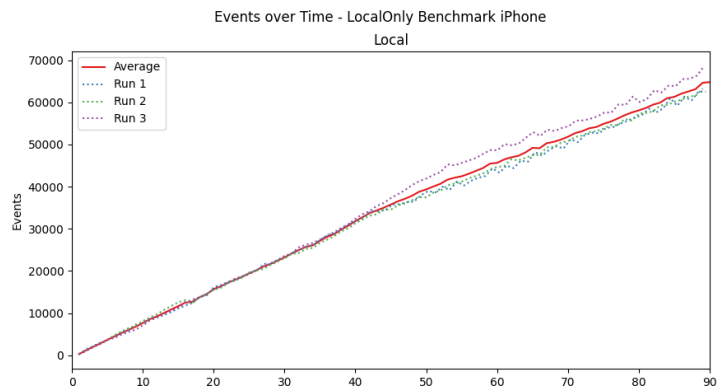


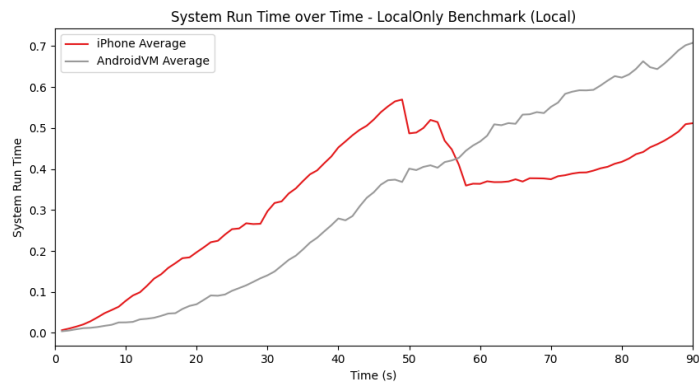
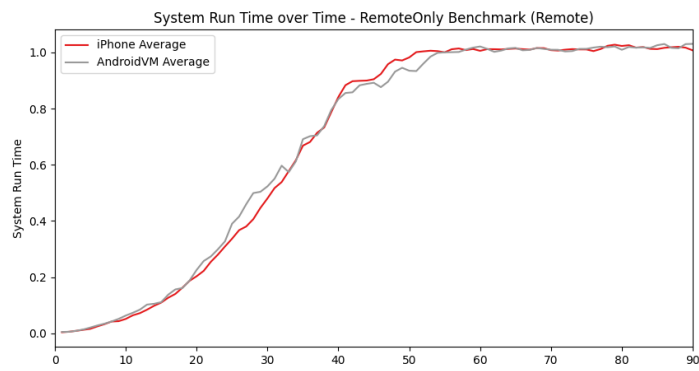
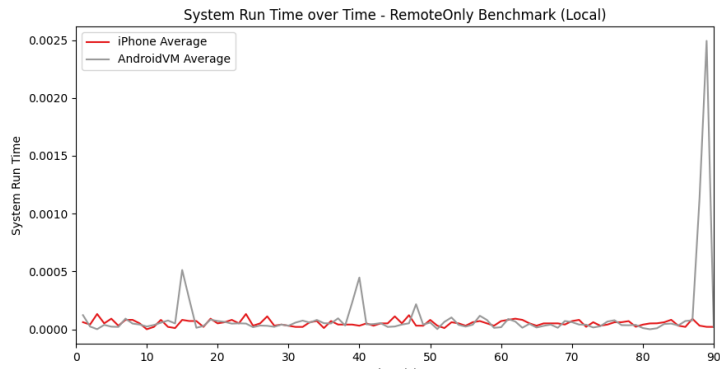
Events over Time - RemoteOnly Benchmark iPhone



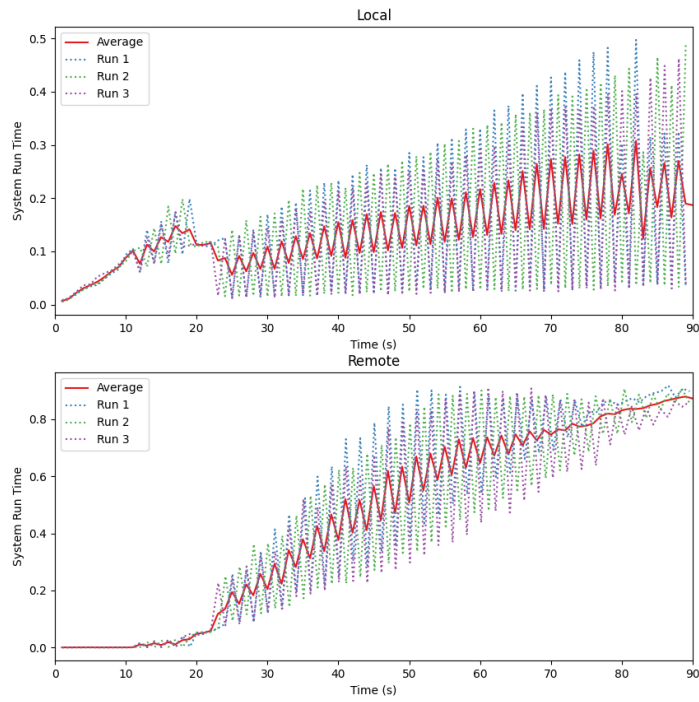
Events over Time - RemoteOnly Benchmark AndroidVM



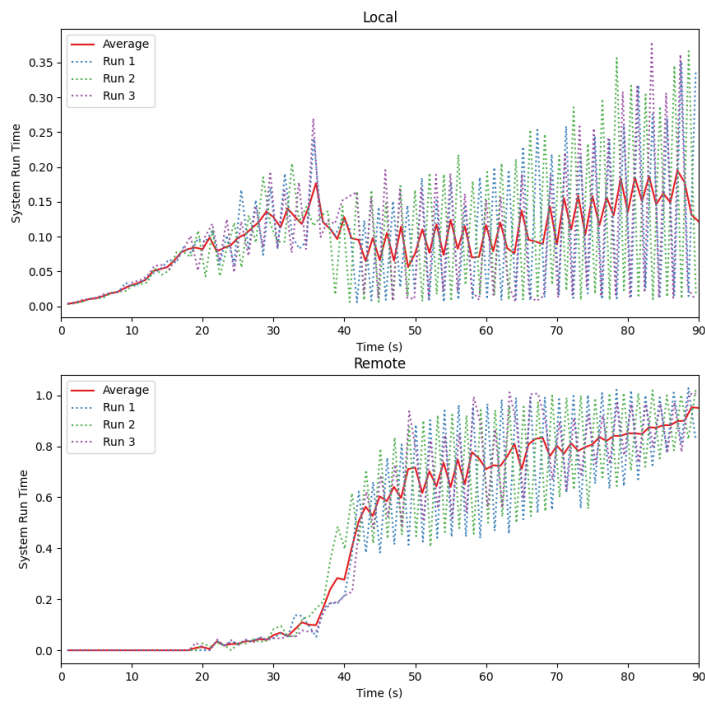




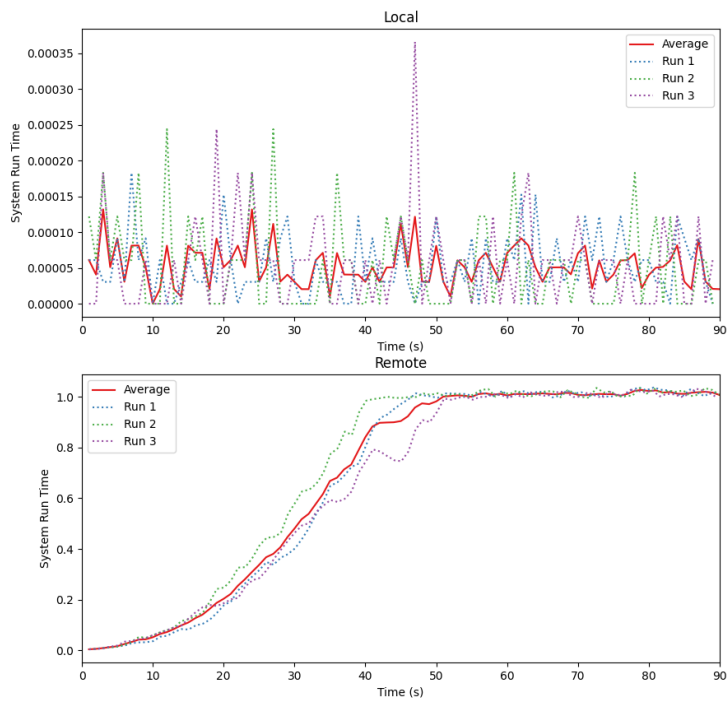
System Run Time over Time - Standard Benchmark iPhone



System Run Time over Time - Standard Benchmark AndroidVM



System Run Time over Time - RemoteOnly Benchmark iPhone



System Run Time over Time - RemoteOnly Benchmark AndroidVM

